

ĐẠI HỌC CÔNG NGHIỆP HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

=====***=====



HaUI

SCHOOL OF INFORMATION
& COMMUNICATIONS TECHNOLOGY

BÁO CÁO BÀI TẬP LỚN THUỘC HỌC PHẦN
HỌC MÁY

ĐỀ TÀI

ỨNG DỤNG CNN, FAISS TÌM KIẾM ẢNH
TƯƠNG ĐỒNG

GVHD	:	ThS. Phạm Việt Anh	
Nhóm	:	05	
Mã lớp	:	20252IT6047003	
Khóa	:	18	
Sinh viên thực hiện	:	Giang Lê Hoàng	2023601729
		Bùi Thùy Dương	2023601608
		Nguyễn Viết Khánh	2023601808
		Nguyễn Tiến Thắng	2023601725

Hà Nội, 2026

ĐẠI HỌC CÔNG NGHIỆP HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

=====***=====



HaUI

SCHOOL OF INFORMATION
& COMMUNICATIONS TECHNOLOGY

BÁO CÁO BÀI TẬP LỚN THUỘC HỌC PHẦN
HỌC MÁY

ĐỀ TÀI

ỨNG DỤNG CNN, FAISS TÌM KIẾM ẢNH
TƯƠNG ĐỒNG

GVHD	:	ThS. Phạm Việt Anh	
Nhóm	:	05	
Mã lớp	:	20252IT6047003	
Khóa	:	18	
Sinh viên thực hiện	:	Giang Lê Hoàng	2023601729
		Bùi Thùy Dương	2023601608
		Nguyễn Viết Khánh	2023601808
		Nguyễn Tiến Thắng	2023601725

Hà Nội, 2026

LỜI NÓI ĐẦU

Ngày nay, dữ liệu hình ảnh ngày càng phổ biến ở nhiều lĩnh vực như mạng xã hội, thương mại điện tử, y tế và an ninh, đặt ra thách thức đối việc quản lý và truy xuất thông tin hiệu quả. Hệ thống tìm kiếm ảnh tương tự (CBIR) ra đời nhằm giải quyết vấn đề này bằng cách tìm, trả về các hình ảnh có nội dung gần giống với ảnh truy vấn.

Tuy nhiên, các phương pháp truyền thống dựa trên đặc trưng thủ công còn hạn chế trong việc nắm bắt ngữ nghĩa phức tạp của hình ảnh. Sự phát triển của học sâu (Deep Learning), đặc biệt là mạng nơ-ron tích chập (CNN), đã mở ra hướng tiếp cận mới nhờ khả năng tự động trích xuất đặc trưng sâu (Deep Features) phản ánh chính xác nội dung hình ảnh.

Từ thực tế đó, nhóm chúng em thực hiện đề tài ***“Ứng dụng CNN, FAISS tìm kiếm ảnh tương đồng”***, với mục tiêu xây dựng mô hình có khả năng tìm kiếm, trả về các ảnh tương đồng dựa trên đặc trưng trích xuất từ mô hình CNN pretrained, kết hợp với thư viện FAISS giúp việc tìm kiếm trở nên nhanh chóng. Các vấn đề cụ thể được trình bày trong báo cáo chia thành 3 chương chính:

Chương 1: Tổng quan đề tài

Chương 2: Cơ sở lý thuyết

Chương 3: Triển khai, đánh giá hệ thống

Đặc biệt, nhóm chúng em xin gửi lời cảm ơn chân thành đến thầy Phạm Việt Anh, người đã trực tiếp hướng dẫn, đồng hành và tạo mọi điều kiện thuận lợi để chúng em có thể hoàn thành bài báo cáo này. Mặc dù thời gian thực hiện đề tài không nhiều, nhưng với sự quyết tâm và nỗ lực của từng thành viên, chúng em đã cùng nhau nghiên cứu, thảo luận và áp dụng kiến thức từ lớp học cũng như các nguồn tài liệu để hoàn thiện bài báo cáo một cách đầy đủ nhất. Vì vậy nhóm chúng em rất mong nhận được những ý kiến đóng góp quý báu của thầy và bạn đọc để báo cáo được hoàn thiện hơn.

Chúng em xin chân thành cảm ơn!

MỤC LỤC

LỜI NÓI ĐẦU	1
MỤC LỤC	2
DANH MỤC HÌNH ẢNH	5
DANH MỤC BẢNG BIỂU	6
CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI.....	7
1.1. Giới thiệu đề tài và phát biểu bài toán	7
1.1.1. Lý do chọn đề tài.....	7
1.1.2. Khảo sát và phát biểu bài toán	7
1.2. Phương pháp giải quyết bài toán.....	9
1.3. Công nghệ sử dụng.....	10
1.4. Phạm vi nghiên cứu.....	11
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT	13
2.1. Tổng quan về hệ thống truy xuất ảnh dựa trên nội dung (CBIR)	13
2.1.1. Khái niệm CBIR.....	13
2.1.2. Quy trình hoạt động của hệ thống CBIR	13
2.1.3. Các thách thức của bài toán	13
2.1.4. Ứng dụng thực tế.....	14
2.2. Mạng nơ-ron tích chập (CNN).....	14
2.2.1. Khái niệm CNN.....	14
2.2.2. Lớp tích chập (Convolution Layer)	15
2.2.3. Hàm kích hoạt ReLU	16
2.2.4. Lớp gộp (Pooling Layer).....	16
2.2.5. Fully Connected Layer.....	17
2.2.6. Ý nghĩa của CNN	17
2.2.7. Các mô hình CNN phổ biến.....	18
2.3. Học chuyển giao (Transfer Learning).....	20

2.3.1. Khái niệm	20
2.3.2. Các phương pháp học chuyển giao	21
2.4. Mô hình ResNet18	21
2.4.1. Vấn đề của mạng sâu.....	21
2.4.2. Residual Learning	22
2.4.3. Kiến trúc ResNet18.....	23
2.5. Độ đo tương đồng Cosine Similarity	26
2.5.1. Khái niệm	26
2.5.2. Công thức	27
2.5.3. Ý nghĩa	27
2.6. Thư viện FAISS.....	28
2.6.1. Giới thiệu FAISS	28
2.6.2. Tìm kiếm Exact và Approximate	28
2.6.3. IndexFlatIP	29
2.6.4. Chuẩn hóa L2	30
2.6.5. Quy trình xây dựng và sử dụng index.....	30
2.6.6. Ưu điểm của FAISS	31
2.7. CLIP (Contrastive Language–Image Pre-training).....	31
2.7.1. Tổng quan.....	31
2.7.2. Kiến trúc cốt lõi của hệ thống CLIP	32
2.7.3. Cơ sở toán học và hàm mất mát (Contrastive Learning)	32
2.7.4. Phân loại Zero-Shot và kỹ thuật điều hướng (Prompt Engineering)	33
2.7.5. Ứng dụng trong AI sinh tạo (Generative AI)	35
2.7.6. Triển khai kỹ thuật thực tiễn, hệ sinh thái API và vi dịch vụ ...	36
CHƯƠNG 3: TRIỂN KHAI, ĐÁNH GIÁ HỆ THỐNG	38
3.1. Môi trường thực nghiệm	38
3.2. Bộ dữ liệu thực nghiệm.....	38

3.2.1. Giới thiệu bộ dữ liệu CIFAR-10	38
3.2.2. Tiền xử lý dữ liệu	39
3.3. Quy trình xây dựng hệ thống	40
3.4. Triển khai hệ thống.....	41
3.4.1. Tổng quan kiến trúc hệ thống	41
3.4.2. Trích xuất đặc trưng bằng ResNet18.....	41
3.4.3. Xây dựng bộ tìm kiếm FAISS	42
3.4.4. Trích xuất đặc trưng bằng CLIP.....	43
3.4.5. Xây dựng Trợ lý Chat.....	44
3.4.6. Xây dựng giao diện web	44
3.5. Các chỉ số đánh giá	45
3.5.1. Precision@K	45
3.5.2. Recall@K	46
3.5.3. Average Precision – AP	46
3.5.4. Tổng hợp ba chỉ số trong hệ thống.....	46
3.6. Kết quả thực nghiệm	47
3.6.1. Cấu trúc thư mục của dự án	47
3.6.2. Kết quả truy xuất ảnh tương đồng.....	47
3.7. Thảo luận kết quả	50
3.8. Kết luận và hướng phát triển.....	52
3.8.1. Kết luận	52
3.8.2. Hướng phát triển	53
TÀI LIỆU THAM KHẢO	55
PHỤ LỤC HƯỚNG DẪN SỬ DỤNG HỆ THỐNG	56

DANH MỤC HÌNH ẢNH

Hình 2.1: Quy trình hoạt động của hệ thống CBIR	13
Hình 2.2: Minh họa lớp convolution.....	16
Hình 2.3: Pooling layer	17
Hình 2.4: Fully connected layer	17
Hình 2.5: AlexNet	19
Hình 2.6: Residual Block	22
Hình 2.7: Cấu trúc của ResNet-18	23
Hình 2.8: Minh họa lớp tiền xử lý.....	24
Hình 2.9: Minh họa 4 stage trong hệ thống residual block.....	24
Hình 2.10: Lớp Global Average Pooling.....	25
Hình 3.1: Cấu trúc thư mục của dự án	47
Hình 3.2: Giao diện màn hình chính khi chạy	47
Hình 3.3: Chọn 1 ảnh để tìm các hình ảnh tương đồng	48
Hình 3.4: Kết quả	48
Hình 3.5: Chat với trợ lý tìm ảnh	49
Hình 3.6: Chat với trợ lý lấy k hình ảnh	49
Hình 3.7: Hỗ trợ ngôn ngữ tiếng anh	50
Hình 3.8: Upload file để tìm kiếm hình ảnh tương đồng.....	50

DANH MỤC BẢNG BIỂU

Bảng 1.1: Các công nghệ, mô hình và bộ dữ liệu sử dụng trong đề tài	10
Bảng 2.1: Cấu trúc 4 stages của ResNet18	25
Bảng 2.2: Ý nghĩa giá trị Cosine trong khoảng $(-1, 1)$	27
Bảng 2.3: Các thư viện và tính năng sử dụng	37
Bảng 3.1: Môi trường thực nghiệm.....	38
Bảng 3.2: Quy trình xây dựng hệ thống.....	40
Bảng 3.3: Cấu trúc frontend.....	45
Bảng 3.4: Chỉ số đánh giá	46

CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI

1.1. Giới thiệu đề tài và phát biểu bài toán

1.1.1. Lý do chọn đề tài

Trong bối cảnh dữ liệu số bùng nổ mạnh mẽ, hình ảnh đã trở thành một trong những dạng dữ liệu phổ biến nhất trên Internet. Mỗi ngày, hàng triệu hình ảnh mới được tạo ra và lưu trữ trên các nền tảng thương mại điện tử, mạng xã hội và hệ thống quản lý dữ liệu. Điều này đặt ra yêu cầu về những phương pháp tìm kiếm thông minh, cho phép người dùng nhanh chóng tìm thấy các hình ảnh có nội dung tương tự từ một ảnh mẫu.

Tuy nhiên, các phương pháp tìm kiếm truyền thống chủ yếu dựa vào từ khóa hoặc thông tin mô tả, vốn phụ thuộc nhiều vào người dùng và khó phản ánh chính xác nội dung trực quan của hình ảnh. Với sự phát triển của trí tuệ nhân tạo, đặc biệt là học sâu (Deep Learning), các mô hình CNN đã cho thấy khả năng trích xuất đặc trưng ảnh hiệu quả, mở ra hướng tiếp cận mới cho bài toán tìm kiếm ảnh tương đồng.

Bên cạnh đó, khi số lượng ảnh ngày càng lớn, việc tìm kiếm trên không gian dữ liệu có kích thước cao đòi hỏi các giải pháp tối ưu về tốc độ và khả năng mở rộng. FAISS là một thư viện tìm kiếm vector hiệu năng cao, được thiết kế để xử lý hiệu quả bài toán tìm kiếm tương đồng trên tập dữ liệu lớn.

Từ những lý do trên, nhóm lựa chọn đề tài ***“Ứng dụng CNN và FAISS trong tìm kiếm ảnh tương đồng”*** nhằm nghiên cứu và xây dựng một hệ thống có khả năng tìm kiếm hình ảnh dựa trên nội dung trực quan, đồng thời tìm hiểu cách kết hợp giữa học sâu và kỹ thuật tìm kiếm vector trong các ứng dụng thực tế.

1.1.2. Khảo sát và phát biểu bài toán

1.1.2.1. Khảo sát thực trạng và tập dữ liệu nghiên cứu

Trong đề tài này, nhóm sử dụng bộ dữ liệu CIFAR-10 để xây dựng và đánh giá hệ thống tìm kiếm ảnh tương đồng. Để định hình giải pháp công nghệ, nhóm tiến hành khảo sát đặc điểm cấu trúc và các thách thức thực tế trên bộ dữ liệu thực nghiệm CIFAR-10:

- Tổng quan quy mô: Bộ dữ liệu gồm 60.000 ảnh màu (RGB), phân chia cân bằng thành 10 lớp đối tượng đối lập (mỗi lớp có 6.000 ảnh).
- Thách thức độ phân giải: Kích thước gốc cực kỳ nhỏ (32x32 pixel), gây mờ nhòe và mất mát các đặc trưng chi tiết (fine-grained features).

- Độ nhiễu loạn thị giác: Tồn tại sự tương đồng thị giác nội lớp cao ở độ phân giải thấp (ví dụ: Chó - Mèo). Điều này đòi hỏi mô hình phải có khả năng trích xuất đặc trưng thô và ngữ nghĩa ở mức trừu tượng cao.
- Cấu trúc kho dữ liệu (Gallery): Đối với bài toán tìm kiếm ảnh (CBIR), hệ thống yêu cầu một kho tra cứu dữ liệu lớn nhất có thể. Do đó, nhóm tiến hành gộp tập Train (50.000 ảnh) và tập Test (10.000 ảnh) thành một kho lưu trữ duy nhất gồm 60.000 ảnh. Việc này tạo ra áp lực tính toán lớn cho thuật toán so khớp lân cận lặp qua không gian đa chiều.
- Nguy cơ Overfitting: Độ phức tạp thấp của dữ liệu hình ảnh (so với ImageNet chẳng hạn) khiến mô hình CNN rất dễ bị quá khớp (overfitting) trên tập huấn luyện nếu kiến trúc mạng quá lớn hoặc thiếu các kỹ thuật chống quá khớp.

1.1.2.2. Phát biểu bài toán

A. Bài toán trích xuất đặc trưng hình ảnh (Feature Extraction)

Cho một kho dữ liệu lưu trữ (Gallery) gồm tập hợp N hình ảnh $D = \{p_1, p_2, \dots, p_N\}$ với $N = 60000$. Khi có một ảnh truy vấn q đầu vào từ người dùng:

- Nhiệm vụ: Xây dựng một hàm ánh xạ phi tuyến $f(x)$ nhằm chuyển đổi một bức ảnh thô dạng pixel thành một vector đặc trưng biểu diễn ngữ nghĩa mật độ cao trong không gian d chiều ($v \in \mathbb{R}^d$).
- Ràng buộc toán học: Vector đặc trưng truy vấn $v_q = f(q)$ và các vector trong cơ sở dữ liệu $v_{p_i} = f(p_i)$ phải bảo toàn được quan hệ ngữ nghĩa thị giác. Tức là, nếu hai bức ảnh q và p_i có nội dung tương đồng nhau về mặt bản chất, thì khoảng cách giữa hai vector v_q và v_{p_i} trong không gian d chiều phải là nhỏ nhất.

B. Bài toán tìm kiếm lân cận và so khớp tương đồng (Similarity Search)

Sau khi chuyển đổi kho ảnh D thành một ma trận đặc trưng kích thước $N \times d$ và tiến hành chuẩn hóa ma trận về độ dài đơn vị (chuẩn $L_2\text{-norm} = 1$) nhằm loại bỏ sự sai khác về độ sáng và độ tương phản, bài toán so khớp được thiết lập:

- Hàm mục tiêu (Đo độ tương đồng): Sử dụng phép tính Tích vô hướng (Inner Product - IP) để đánh giá. Vì các vector đã được chuẩn hóa, tích vô hướng toán học lúc này tương đương với giá trị độ tương đồng Cosine

- Kết quả đầu ra (Top-K): Hệ thống cần thực hiện thuật toán tìm kiếm lân cận để sắp xếp và trích xuất ra một tập hợp gồm K chỉ số ảnh ($K = 10$) từ kho dữ liệu D sao cho điểm tương đồng S_i đạt giá trị lớn nhất.

1.2. Phương pháp giải quyết bài toán

Trong quá trình thực hiện nhóm đã đề xuất 1 số ý tưởng để giải quyết bài toán như sau:

Giải pháp 1: Dùng ResNet pretrained để trích đặc trưng ảnh

- Ý tưởng: dùng ResNet-18 đã huấn luyện sẵn trên ImageNet, bỏ lớp phân loại, chỉ lấy vector đặc trưng để so khớp.
- Ưu điểm: triển khai nhanh, không tốn chi phí huấn luyện, đặc trưng thị giác đủ tốt cho ảnh CIFAR nhỏ, chạy nhẹ trên CPU.
- Nhược điểm: đặc trưng học từ ImageNet, chưa tối ưu riêng cho CIFAR-10.

Giải pháp 2: Fine-tune (huấn luyện lại) model ResNet trên CIFAR-10

- Ý tưởng: huấn luyện lại ResNet trên chính tập CIFAR-10 để đặc trưng "khớp" dữ liệu hơn.
- Ưu điểm: về lý thuyết có thể cho đặc trưng chính xác hơn với đúng miền dữ liệu.
- Nhược điểm: tốn nhiều thời gian và tài nguyên (cần GPU, nhiều epoch), quy trình phức tạp; với ảnh CIFAR 32×32 và 10 lớp thì đặc trưng pretrained đã đủ tốt, mức cải thiện không tương xứng chi phí nên nhóm loại bỏ hướng này.

Giải pháp 3: Chỉ tìm kiếm bằng hình ảnh (không hỗ trợ tìm bằng mô tả văn bản)

- Ý tưởng: hệ thống chỉ nhận ảnh truy vấn, trả về ảnh giống.
- Ưu điểm: đơn giản, chỉ cần một model (ResNet).
- Nhược điểm: hạn chế trải nghiệm - người dùng bắt buộc phải có sẵn ảnh; không tìm được khi chỉ có ý tưởng/mô tả bằng lời (vd "máy bay trên bầu trời") thì chưa đáp ứng đủ nhu cầu thực tế.

Giải pháp 4: Dùng thêm model CLIP để tìm bằng cả ảnh và văn bản mô tả (được chọn)

- Ý tưởng: bổ sung CLIP, nhúng ảnh và văn bản vào cùng một không gian cho phép tìm ảnh bằng câu mô tả, kể cả tiếng Việt (qua bước dịch Việt sang Anh).

- Ưu điểm: mở rộng khả năng truy vấn (ảnh và chữ), hiểu ngữ nghĩa, không cần huấn luyện lại (zero-shot).
- Nhược điểm: thêm một model, nặng hơn đôi chút nhưng vẫn chạy được trên CPU và chấp nhận được.

Nhóm quyết định kết hợp Giải pháp 1 và Giải pháp 4: dùng ResNet-18 pretrained cho tìm bằng ảnh (đặc trưng thị giác, nhẹ, không cần fine-tune) và bổ sung CLIP cho tìm bằng văn bản (đặc trưng ngữ nghĩa), lập chỉ mục bằng FAISS để tăng tốc. Phương án này tận dụng ưu điểm của cả hai model, khắc phục hạn chế "chỉ tìm bằng ảnh" của Giải pháp 3 và tránh chi phí không cần thiết của việc fine-tune ở Giải pháp 2.

1.3. Công nghệ sử dụng

Bảng 1.1: Các công nghệ, mô hình và bộ dữ liệu sử dụng trong đề tài

Thành phần	Nguồn/Thư viện	Vai trò trong hệ thống
Python	Python 3.12	Ngôn ngữ lập trình chính để xây dựng toàn bộ hệ thống.
PyTorch	Meta AI	Hỗ trợ tải và sử dụng mô hình học sâu phục vụ trích xuất đặc trưng ảnh.
ResNet18	Torchvision (ImageNet Pre-trained)	Trích xuất ảnh đầu vào kích thước 224×224 thành vector đặc trưng 512 chiều.
FAISS	Meta AI	Xây dựng chỉ mục IndexFlatIP và thực hiện tìm kiếm vector hiệu năng cao.
NumPy	Python Package	Lưu trữ và xử lý vector đặc trưng (features.npy), hỗ trợ các phép toán trên ma trận và vector.
Flask	Python Package	Xây dựng API phục vụ giao tiếp giữa các thành phần của hệ thống.
CIFAR-10	Dataset Benchmark	Bộ dữ liệu thực nghiệm gồm 60.000 ảnh thuộc 10 lớp đối tượng khác nhau.

OpenCLIP	LAION	Nhúng ảnh và văn bản vào cùng một không gian vector, phục vụ tìm kiếm ảnh bằng mô tả chữ, hỗ trợ tiếng Việt.
HTML / CSS / JavaScript	Frontend thuần (Vanilla JS)	Xây dựng giao diện người dùng: upload ảnh, khung chat, hiển thị kết quả và chỉ số đánh giá.
Docker & Docker Compose	Docker Inc.	Đóng gói toàn bộ hệ thống thành container, đảm bảo chạy nhất quán trên mọi máy không cần cài thủ công.
uv	Astral	Quản lý môi trường ảo và phiên bản thư viện Python (thay thế pip/requirements.txt), khai báo qua pyproject.toml + uv.lock.

1.4. Phạm vi nghiên cứu

Đề tài tập trung nghiên cứu và xây dựng hệ thống tìm kiếm ảnh tương đồng dựa trên đặc trưng học sâu và kỹ thuật tìm kiếm vector hiệu năng cao. Trong đó, mô hình ResNet-18 và CLIP tiền huấn luyện được sử dụng để trích xuất đặc trưng ảnh, kết hợp với thư viện FAISS nhằm tăng tốc quá trình truy xuất trên tập dữ liệu có kích thước lớn.

Nội dung thực hiện

- Nghiên cứu tổng quan về bài toán tìm kiếm ảnh tương đồng (Content-Based Image Retrieval - CBIR).
- Khảo sát các phương pháp trích xuất đặc trưng ảnh và lựa chọn mô hình phù hợp.
- Sử dụng mô hình ResNet18 đã được huấn luyện trước để trích xuất vector đặc trưng từ ảnh đầu vào.
- Xây dựng cơ chế lập chỉ mục và tìm kiếm lân cận bằng thư viện FAISS.
- Xây dựng ứng dụng cho phép người dùng tải ảnh truy vấn và hiển thị các ảnh tương tự được tìm thấy.

Giới hạn của đề tài

- Đề tài sử dụng mô hình ResNet18 tiền huấn luyện và không thực hiện huấn luyện lại (fine-tuning) trên tập dữ liệu riêng.
- Việc tìm kiếm ảnh được thực hiện trên bộ dữ liệu thực nghiệm CIFAR-10 với quy mô 60.000 ảnh.

- Đề tài chưa triển khai trên môi trường điện toán đám mây hoặc hệ thống phân tán quy mô lớn.
- Các phương pháp tìm kiếm xấp xỉ nâng cao của FAISS như IVF hoặc Product Quantization (PQ) chỉ được đề cập như hướng phát triển trong tương lai và chưa được tích hợp vào hệ thống hiện tại.

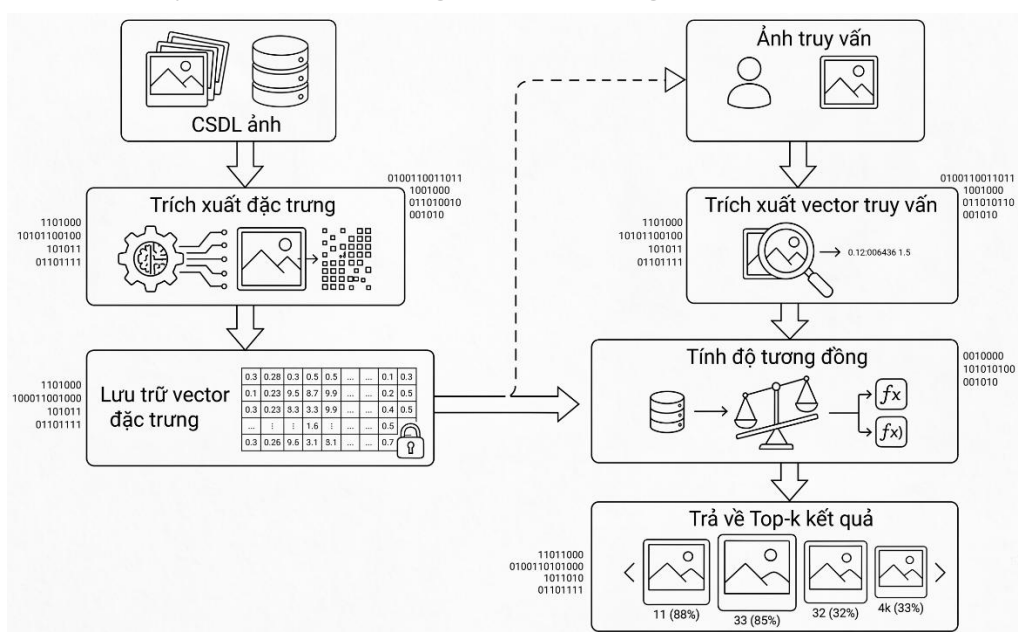
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. Tổng quan về hệ thống truy xuất ảnh dựa trên nội dung (CBIR)

2.1.1. Khái niệm CBIR

Hệ thống tìm kiếm ảnh dựa trên nội dung (Content-Based Image Retrieval - CBIR) là kỹ thuật truy xuất thông tin thị giác bằng cách phân tích trực tiếp nội dung thực tế của bức ảnh (như màu sắc, hình khối, kết cấu hoặc ngữ nghĩa) thay vì phụ thuộc vào các nhãn mô tả bằng văn bản do con người gán vào.

2.1.2. Quy trình hoạt động của hệ thống CBIR



Hình 2.1: Quy trình hoạt động của hệ thống CBIR

2.1.3. Các thách thức của bài toán

Để xây dựng một hệ thống tìm kiếm ảnh hiệu quả, cần giải quyết 3 thách thức lớn sau:

- Khoảng cách ngữ nghĩa (Semantic gap): Máy tính chỉ hiểu giá trị pixel thô (độ sáng, màu sắc), trong khi con người hiểu nội dung cao cấp (bối cảnh, đối tượng). Hệ thống phải làm sao để máy tính hiểu được "ngữ nghĩa" của bức ảnh.
- Quy mô dữ liệu lớn: Kho lưu trữ thực tế chứa hàng trăm nghìn đến hàng triệu bức ảnh. Việc lưu trữ và xử lý một lượng vector đặc trưng khổng lồ trên RAM rất dễ gây quá tải hệ thống.
- Tốc độ truy xuất: Người dùng luôn đòi hỏi kết quả tìm kiếm phải trả về ngay lập tức (thời gian thực - Real-time). Khi dữ liệu tăng, các thuật toán tìm kiếm thông thường sẽ bị chậm và gây trễ lớn.

2.1.4. Ứng dụng thực tế

Hệ thống tìm kiếm ảnh dựa trên nội dung (CBIR) không còn là lý thuyết mà đã trở thành công nghệ cốt lõi, vận hành các siêu hệ thống với quy mô dữ liệu khổng lồ:

- Mua sắm bằng hình ảnh (Taobao): Tìm kiếm trong kho 3 tỷ ảnh sản phẩm. Hệ thống giải quyết 10.000 ảnh/giây, trả kết quả tương tự trong 0.1 giây.
- Chẩn đoán y tế (Google Health): Quét kho dữ liệu 130.000 ảnh chụp y tế. Tìm ca bệnh tương đồng chính xác 98.1% chỉ trong 2 giây.
- Tìm kiếm ảnh cá nhân (Google Photos): Tự động nhận diện bối cảnh, khuôn mặt cho 4 tỷ ảnh mới mỗi ngày. Tìm lại ảnh cũ chỉ mất 0.05 giây.
- Nhận diện an ninh (Amazon Rekognition): Đối chiếu ảnh camera với kho căn cước hàng chục triệu người. Độ chính xác đạt 99.8%, truy vết đối tượng dưới 0.5 giây.

2.2. Mạng nơ-ron tích chập (CNN)

2.2.1. Khái niệm CNN

Thuật toán CNN, hay còn gọi Convolutional Neural Network (Mạng Nơ-ron Tích Chập). Đây là một loại mô hình học sâu rất mạnh mẽ trong lĩnh vực trí tuệ nhân tạo, đặc biệt là trong xử lý hình ảnh. CNN giúp xây dựng các hệ thống thông minh với độ chính xác cao, nhờ khả năng nhận diện và phân tích các đặc điểm quan trọng trong ảnh.

CNN hoạt động bằng cách xử lý dữ liệu hình ảnh thông qua các lớp tích chập, giúp trích xuất các đặc điểm nổi bật từ hình ảnh một cách hiệu quả. Ví dụ, thuật toán này thường được sử dụng để nhận diện khuôn mặt, phân loại đối tượng, và nhiều ứng dụng khác. Những nền tảng nổi tiếng như Facebook và Google cũng đã tích hợp CNN để cải thiện khả năng nhận diện hình ảnh trên các dịch vụ.

Về mặt kỹ thuật, khi một hình ảnh được đưa vào hệ thống CNN, nó sẽ trải qua một loạt các bước. Đầu tiên, hình ảnh sẽ được xử lý qua các lớp tích chập với các bộ lọc để trích xuất các đặc điểm. Sau đó, dữ liệu sẽ đi qua các lớp kết nối đầy đủ và cuối cùng là lớp phân loại sử dụng hàm Softmax. Nó đưa ra xác suất cho các loại đối tượng khác nhau. Kết quả cuối cùng sẽ cho chúng ta biết khả năng thuộc về từng loại của đối tượng trong hình ảnh.

Mạng nơ-ron tích chập gồm 3 thành phần chính:

- Các lớp tích chập (convolution)
- Lớp pooling

- Lớp kết nối đầy đủ

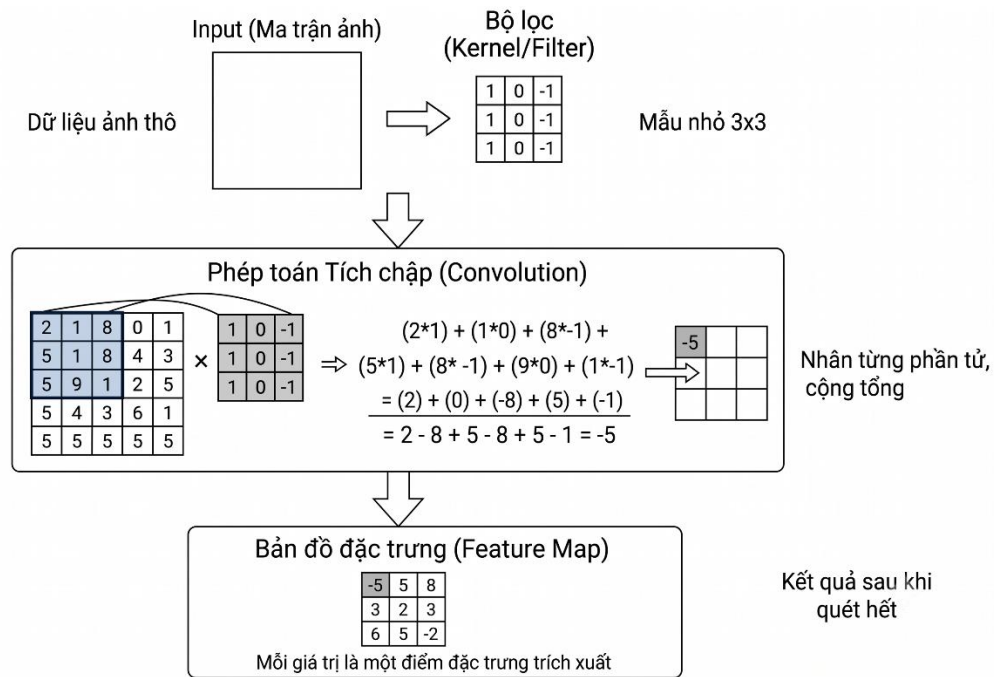
Các siêu tham số quan trọng của lớp tích chập

- Kích thước bộ lọc (Kernel size): Thường là 3×3 , 5×5 , 7×7 . Bộ lọc nhỏ giúp nắm bắt các chi tiết tinh vi cục bộ, trong khi bộ lọc lớn nhìn thấy vùng rộng hơn nhưng bỏ qua chi tiết nhỏ.
- Bước nhảy (Stride): Là khoảng cách (tính bằng pixel) mà bộ lọc dịch chuyển sau mỗi lần tính toán. Stride = 1 nghĩa là bộ lọc nhích từng pixel, giữ nguyên độ chi tiết. Stride = 2 nghĩa là bộ lọc nhảy cóc, làm giảm kích thước bản đồ đặc trưng đi một nửa, giúp giảm khối lượng tính toán.
- Lề (Padding): Khi bộ lọc trượt ở rìa bức ảnh, nó có thể bị thiếu dữ liệu. Padding là kỹ thuật thêm các pixel giả (thường có giá trị 0) bao quanh bức ảnh. "Zero-padding" giúp giữ nguyên kích thước không gian của ảnh sau khi tích chập, cho phép xây dựng các mạng rất sâu mà không bị co ngót hình ảnh về 0.

2.2.2. Lớp tích chập (Convolution Layer)

Đây là thành phần quan trọng nhất của mạng CNN, chịu trách nhiệm tự động trích xuất các đặc trưng thị giác từ bức ảnh đầu vào.

- Khái niệm Kernel/Filter: Là một ma trận số nhỏ (thường có kích thước 3×3 hoặc 5×5) chứa các trọng số toán học. Mỗi bộ lọc được thiết kế để phát hiện một đặc điểm hình học cụ thể (như cạnh ngang, cạnh dọc, góc hoặc màu sắc).
- Quá trình trượt cửa sổ (Sliding Window): Bộ lọc sẽ trượt tuần tự qua từng vùng pixel của bức ảnh đầu vào từ trái sang phải, từ trên xuống dưới. Tại mỗi vị trí dừng lại, hệ thống thực hiện phép tính tích vô hướng (Dot Product) giữa các hệ số của bộ lọc và giá trị pixel nằm bên dưới nó.
- Bản đồ đặc trưng (Feature Map): Kết quả của quá trình trượt bộ lọc tạo ra một ma trận mới gọi là bản đồ đặc trưng. Những vị trí nào trên ảnh khớp với đặc điểm mà bộ lọc tìm kiếm sẽ có giá trị số rất cao trên bản đồ này.



Hình 2.2: Minh họa lớp convolution

2.2.3. Hàm kích hoạt ReLU

Ngay sau mỗi phép toán tích chập, dữ liệu sẽ được đưa qua hàm kích hoạt phi tuyến ReLU (Rectified Linear Unit).

- Công thức toán học:

$$f(x) = \max(0, x)$$

Hàm số này hoạt động theo cơ chế đơn giản: giữ nguyên các giá trị dương và thay thế toàn bộ các giá trị âm bằng số 0.

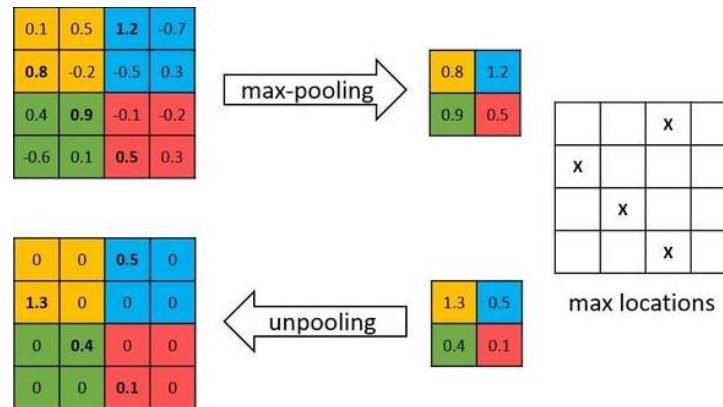
- Vai trò tạo tính phi tuyến: Bản chất của phép tính tích chập là các phép toán tuyến tính. Tuy nhiên, thế giới hình ảnh thực tế lại mang tính phi tuyến tính phức tạp (như sự thay đổi đột ngột của ánh sáng, góc khuất). Hàm ReLU giúp phá vỡ tính tuyến tính, cho phép mạng nơ-ron có khả năng học và biểu diễn được các mẫu hình ảnh phức tạp và trừu tượng.

2.2.4. Lớp gộp (Pooling Layer)

Lớp gộp thường được chèn xen kẽ giữa các lớp tích chập để đơn giản hóa lượng thông tin của bản đồ đặc trưng.

- Max Pooling: Chia bản đồ đặc trưng thành các vùng nhỏ (thường là 2×2) và chỉ giữ lại giá trị lớn nhất trong vùng đó. Đây là phương pháp phổ biến nhất vì nó giúp giữ lại các đặc trưng thị giác mạnh nhất.
- Average Pooling: Thay vì lấy giá trị lớn nhất, phương pháp này tính giá trị trung bình của tất cả các phần tử trong vùng ô quét.

- Giảm số chiều dữ liệu: Lớp gộp giúp giảm kích thước không gian của ma trận ảnh (chiều rộng và chiều cao). Việc này làm giảm đáng kể số lượng tham số cần tính toán, giúp mạng chạy nhanh hơn và kiểm soát tốt hiện tượng quá khớp (Overfitting).

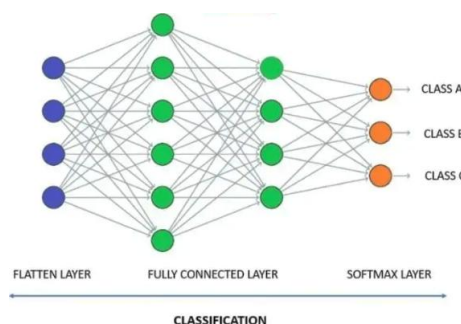


Hình 2.3: Pooling layer

2.2.5. Fully Connected Layer

Lớp kết nối đầy đủ nằm ở giai đoạn cuối cùng của cấu trúc mạng CNN truyền thống.

- Tổng hợp đặc trưng: Sau khi ảnh đi qua nhiều lớp tích chập và lớp gộp, ma trận dữ liệu được duỗi thẳng thành một vector một chiều (Flattening). Lớp FC sẽ kết nối mọi nơ-ron của vector này lại với nhau để tổng hợp toàn bộ các đặc trưng cục bộ đã học được từ các tầng trước đó.
- Phân loại: Lớp FC đóng vai trò như một bộ đưa ra quyết định cuối cùng, dịch dữ liệu đặc trưng thành các điểm số xác suất để phân loại bức ảnh thuộc về nhãn đối tượng nào.



Hình 2.4: Fully connected layer

2.2.6. Ý nghĩa của CNN

Tự động trích xuất đặc trưng

Trước đây, khi xử lý ảnh, người ta phải tự xây dựng các đặc trưng như cạnh, góc, màu sắc, kết cấu,...CNN có thể tự học các đặc trưng này từ dữ liệu.

Ví dụ:

- Lớp đầu tiên học các cạnh và đường nét.
- Lớp giữa học các hình dạng đơn giản như mắt, mũi, bánh xe.
- Lớp sâu hơn học các đối tượng hoàn chỉnh như khuôn mặt, ô tô, động vật.

Giảm số lượng tham số

CNN sử dụng bộ lọc (kernel/filter) được chia sẻ trên toàn bộ ảnh thay vì kết nối đầy đủ như mạng ANN truyền thống. Điều này:

- Giảm đáng kể số lượng tham số cần học.
- Giảm chi phí tính toán.
- Hạn chế hiện tượng overfitting.

Khai thác thông tin không gian của ảnh

CNN giữ được mối quan hệ vị trí giữa các điểm ảnh lân cận thông qua phép tích chập. Nhờ đó mô hình hiểu được cấu trúc của hình ảnh tốt hơn.

Tăng khả năng nhận dạng đối tượng

Nhờ các lớp tích chập và pooling, CNN có thể nhận diện một đối tượng ngay cả khi:

- Đối tượng thay đổi vị trí.
- Kích thước khác nhau.
- Có nhiều hoặc bị che khuất một phần.

2.2.7. Các mô hình CNN phổ biến

Mạng nơ-ron tích chập (CNN) đã có nhiều cải tiến đáng kể từ khi ra đời, giúp nâng cao hiệu suất nhận diện hình ảnh và xử lý thị giác máy tính. Nhiều Convolutional Neural Network Architecture (mô hình CNN) đã được phát triển nhằm tối ưu hóa độ chính xác, giảm thiểu số lượng tham số và tăng tốc độ huấn luyện.

Lenet

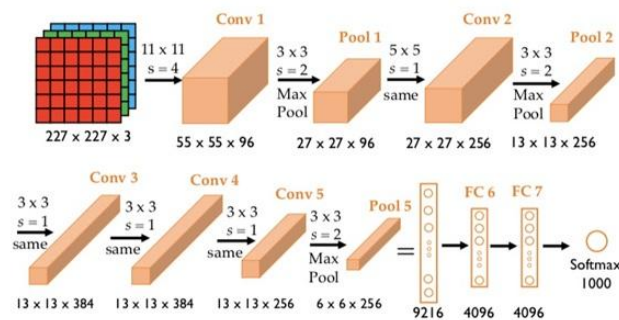
LeNet là một trong những mạng nơ-ron tích chập đầu tiên, được phát triển bởi Yann LeCun vào cuối những năm 1989. Mô hình này được thiết kế để nhận diện chữ số viết tay trên tập dữ liệu MNIST, đặt nền tảng quan trọng cho các kiến trúc CNN sau này. Cấu trúc của LeNet bao gồm các lớp tích chập xen kẽ với các lớp pooling, tiếp theo là các lớp kết nối đầy đủ để thực hiện phân loại. Mặc dù

LeNet có độ sâu tương đối nhỏ so với các mô hình hiện đại, nhưng nó đã chứng minh hiệu quả của CNN trong việc trích xuất đặc trưng từ hình ảnh và xử lý dữ liệu thị giác.

AlexNet

AlexNet được phát triển bởi Alex Krizhevsky, Ilya Sutskever và Geoffrey Hinton vào năm 2012, là Convolutional Neural Network Architecture đầu tiên giành chiến thắng trong cuộc thi ImageNet Large Scale Visual Recognition Challenge (ILSVRC). Đây là bước ngoặt quan trọng giúp Convolutional Neural Networks trở thành công cụ mạnh mẽ trong lĩnh vực nhận diện hình ảnh.

Cấu trúc của AlexNet bao gồm năm lớp tích chập, xen kẽ với ba lớp pooling, sau đó là ba lớp kết nối đầy đủ. Một điểm đột phá của mô hình này là việc sử dụng hàm kích hoạt ReLU, giúp tăng tốc độ huấn luyện đáng kể so với các mô hình trước đó. Ngoài ra, AlexNet cũng áp dụng Dropout để giảm overfitting và sử dụng GPU để tăng tốc tính toán. Thành công của AlexNet đã mở đường cho nhiều mô hình CNN hiện đại sau này.



Hình 2.5: AlexNet

Resnet

ResNet (Residual Network), được giới thiệu bởi nhóm nghiên cứu của Microsoft vào năm 2015, là một mạng nơ-ron tích chập (CNN) với khả năng huấn luyện các mô hình cực kỳ sâu mà không gặp phải vấn đề suy giảm đạo hàm (vanishing gradient).

Điểm khác biệt chính của ResNet là việc sử dụng skip connections (kết nối tắt), cho phép các thông tin có thể đi qua một số lớp nhất định mà không bị suy giảm. Điều này giúp mạng học được các biểu diễn tốt hơn và cải thiện khả năng tổng quát hóa. ResNet có nhiều phiên bản khác nhau, từ ResNet-18, ResNet-34, ResNet-50 đến ResNet-152, trong đó phiên bản sâu nhất đạt đến 152 lớp. Mô hình này đã giành chiến thắng trong cuộc thi ImageNet 2015 với độ lỗi chỉ 3,57%, thấp

nhất trong lịch sử cuộc thi bấy giờ và thậm chí là thấp hơn cả mức nhận diện của con người.

GoogleNet

Khi tìm hiểu về convolutional neural networks là gì? Chắc hẳn bạn sẽ bắt gặp GoogleNet, còn được gọi là Inception v1. Mô hình này được phát triển bởi Google và giành chiến thắng trong cuộc thi ImageNet Challenge năm 2014. Đây là một mô hình Convolutional Neural Network nổi bật với khả năng đạt hiệu suất cao trong phân loại hình ảnh mà không cần sử dụng quá nhiều tham số.

GoogleNet sử dụng Inception Modules, một cơ chế giúp mô hình có thể học được nhiều đặc trưng ở các mức độ khác nhau bằng cách áp dụng các bộ lọc có kích thước khác nhau trong cùng một lớp. Tổng cộng mô hình CNN này có 22 lớp (nếu tính cả các lớp gộp là 27), nhưng số lượng tham số ít hơn giúp giảm tải bộ nhớ và tăng tốc độ huấn luyện mà vẫn đạt độ chính xác cao.

VGG

VGG (Visual Geometry Group), được phát triển bởi nhóm nghiên cứu tại Đại học Oxford, là một mạng nơ-ron tích chập (CNN) sử dụng các bộ lọc 3×3 được xếp chồng lên nhau, tạo nên một kiến trúc đơn giản nhưng rất hiệu quả. Hai phiên bản phổ biến nhất là VGG-16 và VGG-19, với số lượng lớp tích chập lên đến 16 và 19 lớp tương ứng.

Mặc dù VGG đạt được kết quả rất tốt trong nhận diện hình ảnh, nhưng nó có một nhược điểm là số lượng tham số rất lớn (khoảng 138 triệu tham số), khiến mô hình trở nên nặng nề và tốn tài nguyên tính toán. Tuy nhiên, nhờ cấu trúc đồng nhất và khả năng học sâu, VGG vẫn được sử dụng rộng rãi và làm nền tảng cho nhiều nghiên cứu tiếp theo trong thị giác máy tính.

2.3. Học chuyển giao (Transfer Learning)

2.3.1. Khái niệm

Học chuyển giao (Transfer Learning) là kỹ thuật tái sử dụng tri thức mà mô hình đã học được từ một bài toán nguồn để áp dụng cho một bài toán đích mới có liên quan. Phương pháp này giúp giảm chi phí tính toán, rút ngắn thời gian huấn luyện và nâng cao hiệu quả mô hình khi dữ liệu huấn luyện của bài toán đích không đủ lớn.

2.3.2. Các phương pháp học chuyển giao

Tùy thuộc vào mức độ tương đồng giữa bài toán nguồn và bài toán đích, học chuyển giao có thể được thực hiện theo nhiều phương pháp khác nhau. Một số phương pháp phổ biến bao gồm:

a) Sử dụng mô hình tiền huấn luyện làm bộ trích xuất đặc trưng (Feature Extraction)

Trong phương pháp này, mô hình đã được huấn luyện trước trên một tập dữ liệu lớn được sử dụng để trích xuất các đặc trưng từ dữ liệu của bài toán đích. Các lớp tích chập (convolutional layers) được giữ nguyên, trong khi chỉ huấn luyện lại bộ phân loại ở các lớp cuối. Phương pháp này phù hợp khi tập dữ liệu đích có kích thước nhỏ và có tính chất tương đối giống với dữ liệu nguồn.

b) Tinh chỉnh mô hình (Fine-tuning)

Fine-tuning là quá trình tiếp tục huấn luyện một phần hoặc toàn bộ mô hình tiền huấn luyện trên tập dữ liệu đích. Thông thường, các lớp đầu của mạng được giữ nguyên do đã học được những đặc trưng tổng quát, còn các lớp cuối được cập nhật để thích nghi với đặc điểm của bài toán mới. Phương pháp này thường mang lại hiệu quả cao hơn so với chỉ sử dụng trích xuất đặc trưng, đặc biệt khi có đủ dữ liệu huấn luyện.

c) Huấn luyện lại hoàn toàn (Full Training)

Trong trường hợp bài toán đích có tập dữ liệu lớn và khác biệt đáng kể so với bài toán nguồn, toàn bộ mô hình có thể được huấn luyện lại từ đầu hoặc khởi tạo bằng trọng số của mô hình tiền huấn luyện rồi cập nhật tất cả các tham số. Tuy nhiên, phương pháp này đòi hỏi nhiều tài nguyên tính toán và thời gian huấn luyện hơn.

Việc lựa chọn phương pháp học chuyển giao phù hợp phụ thuộc vào quy mô dữ liệu đích, mức độ tương đồng giữa hai miền dữ liệu và yêu cầu về hiệu năng của hệ thống.

2.4. Mô hình ResNet18

2.4.1. Vấn đề của mạng sâu

Thông thường, việc tăng số lượng lớp trong mạng CNN giúp mô hình học được nhiều đặc trưng phức tạp hơn, từ đó cải thiện khả năng nhận dạng và phân loại ảnh. Tuy nhiên, khi mạng trở nên quá sâu, quá trình huấn luyện sẽ gặp nhiều khó khăn.

Trước hết, trong quá trình lan truyền ngược, tín hiệu gradient có thể bị suy giảm hoặc tăng quá mức khi đi qua nhiều lớp liên tiếp. Điều này khiến các lớp ở đầu mạng khó cập nhật trọng số hiệu quả, làm cho việc học diễn ra chậm hoặc không ổn định.

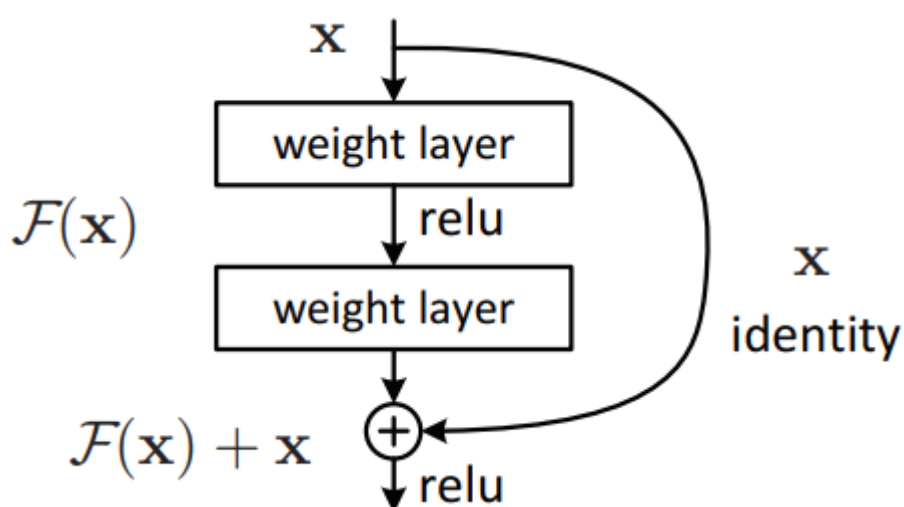
Ngoài ra, việc bổ sung thêm nhiều lớp không phải lúc nào cũng giúp mô hình hoạt động tốt hơn. Trên thực tế, sau một mức độ sâu nhất định, mạng có thể xuất hiện hiện tượng suy giảm hiệu năng (Degradation Problem), tức là mô hình càng sâu thì kết quả huấn luyện và kiểm thử lại càng kém. Nói cách khác, mạng không tận dụng được lợi ích của việc tăng số lớp mặc dù về lý thuyết nó có khả năng học mạnh hơn.

Bên cạnh đó, các mạng sâu thường đòi hỏi nhiều tài nguyên tính toán, thời gian huấn luyện dài và lượng dữ liệu lớn để hạn chế hiện tượng quá khớp.

Để khắc phục những hạn chế trên, kiến trúc Residual Network (ResNet) đã được đề xuất. Điểm nổi bật của ResNet là sử dụng các kết nối tắt (skip connections), cho phép thông tin từ đầu vào được truyền trực tiếp đến các lớp phía sau. Nhờ đó, mô hình học dễ dàng hơn, quá trình lan truyền gradient được cải thiện và việc huấn luyện các mạng có độ sâu lớn trở nên hiệu quả hơn.

2.4.2. Residual Learning

Residual Learning (học phần dư) là cơ chế cốt lõi được đề xuất trong kiến trúc ResNet nhằm khắc phục hiện tượng suy giảm hiệu năng khi độ sâu của mạng nơ-ron tăng lên. Thay vì để các lớp trong mạng học trực tiếp một ánh xạ mong muốn ($H(x)$) từ đầu vào (x), Residual Learning cho phép mạng học phần chênh lệch giữa đầu vào và đầu ra, được gọi là hàm dư (residual function).



Hình 2.6: Residual Block

Residual Block là một khối xử lý trong mạng ResNet, trong đó:

Công thức: $H(x) = F(x) + x$

- $F(x)$: phần mạng cần học (thường gồm 2 lớp Conv + BatchNorm + ReLU)
- x : đầu vào được “nối tắt” sang output (skip connection).

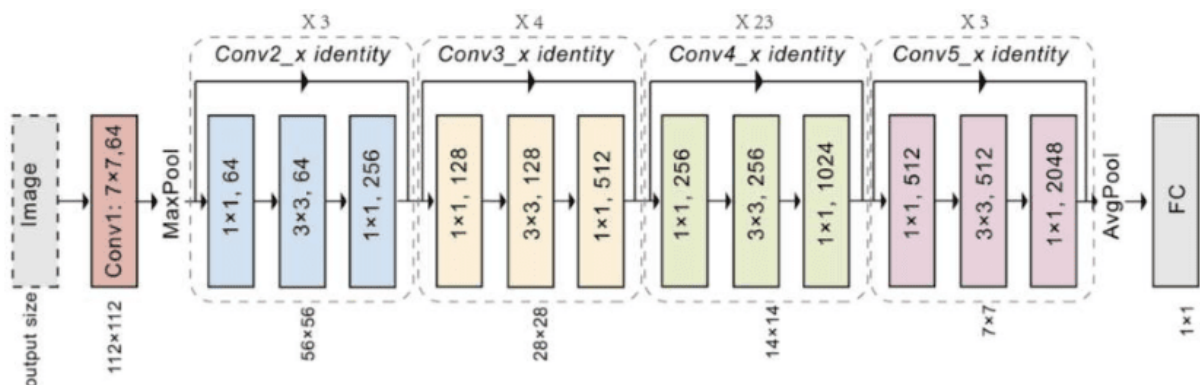
Ý nghĩa:

- Nếu các lớp mới không học được gì tốt $\Rightarrow$ mô hình vẫn có thể sao chép đầu vào vì có thành phần “+ x ”.
- Gradient cũng đi ngược dễ hơn qua đường tắt $\Rightarrow$ giảm hiện tượng vanishing gradient.
- Nhờ vậy, ResNet có thể tăng số lớp mà không làm giảm hiệu suất, trái ngược với CNN truyền thống.
- Residual Block chính là lý do ResNet trích xuất đặc trưng ảnh rất tốt.

2.4.3. Kiến trúc ResNet18

ResNet-18 là một trong những phiên bản nhỏ và nhẹ của họ ResNet. Tên “18” có nghĩa là toàn bộ mô hình có 18 lớp có trọng số (17 lớp tích chập và 1 lớp kết nối đầy đủ), không tính những lớp thao tác như ReLU hay pooling. Mặc dù nhỏ gọn, ResNet-18 vẫn đủ mạnh để trích xuất đặc trưng ảnh chất lượng cao, nên rất phù hợp cho các hệ thống tìm kiếm ảnh tương tự.

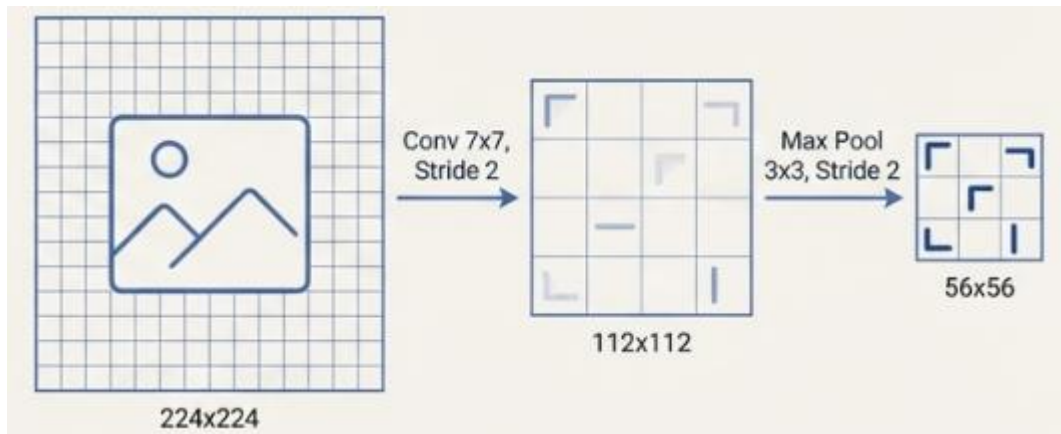
Tổng thể kiến trúc ResNet-18 gồm 4 phần lớn:



Hình 2.7: Cấu trúc của ResNet-18

2.4.3.1. Lớp tiền xử lý ban đầu - Conv1

Khi ảnh được đưa vào ResNet-18, mô hình không cho ảnh đi thẳng vào các Residual Block ngay lập tức. Thay vào đó, nó có một bộ xử lý mở đầu để chuẩn bị ảnh cho phần mạng sâu phía sau.



Hình 2.8: Minh họa lớp tiền xử lý

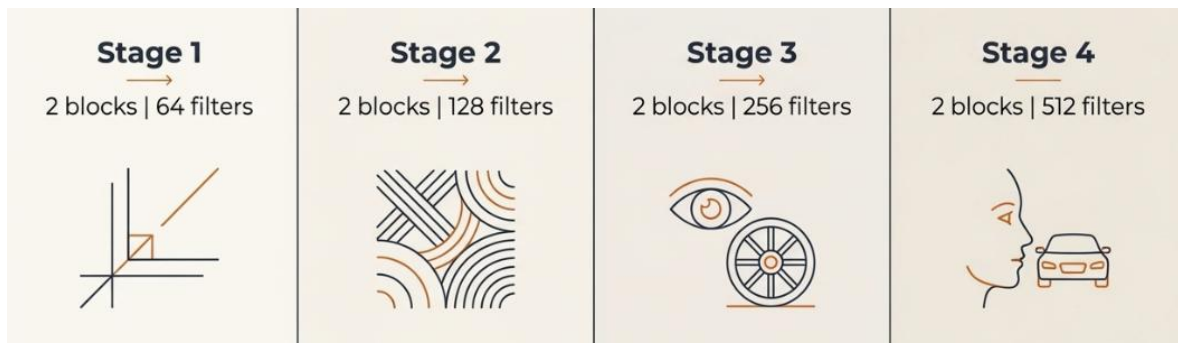
Khối này gồm:

- Tích chập 7×7 , 64 filters, stride 2 $\rightarrow$ quét ảnh với vùng nhìn rộng, tạo 64 kênh đặc trưng và giảm kích thước xuống $1/2$.
- BatchNorm + ReLU $\rightarrow$ ổn định phân phối và tăng tính phi tuyến.
- MaxPooling 3×3 , stride 2 $\rightarrow$ tiếp tục giảm kích thước và giữ lại đặc trưng mạnh nhất.

Nhờ chuỗi bước này, ảnh đầu vào từ 224×224 được giảm xuống khoảng 56×56 , trong khi các đặc trưng cơ bản như cạnh, viền, vùng sáng tối đã được trích xuất đầy đủ. Điều này giúp những block phía sau xử lý nhẹ nhàng hơn và đảm bảo mô hình hoạt động ổn định, hiệu quả.

2.4.3.2. Bốn stage chính - hệ thống residual blocks

Đây là “xương sống” của mạng ResNet-18. Tổng cộng mô hình có 4 stage, mỗi stage gồm 2 residual block, và số lượng filters tăng dần để học đặc trưng phức tạp hơn.



Hình 2.9: Minh họa 4 stage trong hệ thống residual block

Bảng 2.1: Cấu trúc 4 stages của ResNet18

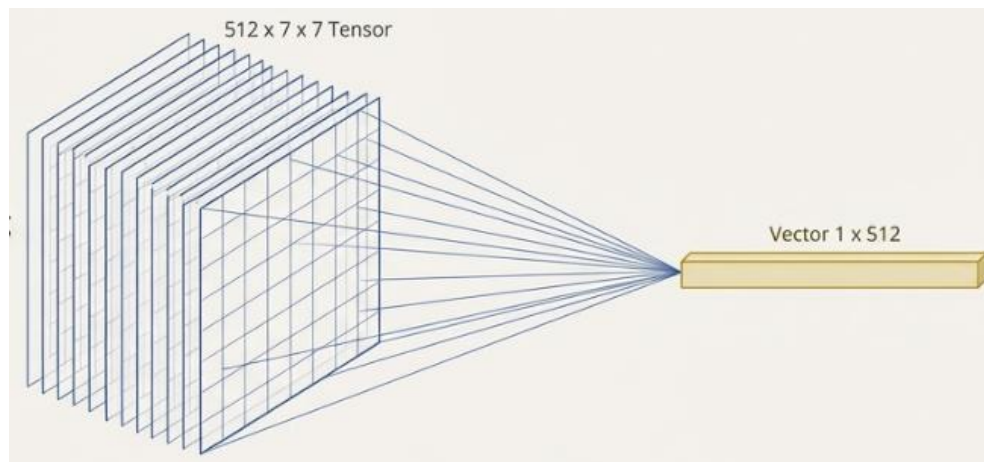
Stage	Số block	Số filter	Ý nghĩa
Stage 1	2 blocks	64	Học đặc trưng đơn giản (cạnh, góc)
Stage 2	2 blocks	128	Học đặc trưng trung gian (đường cong, họa tiết)
Stage 3	2 blocks	256	Học hình dạng lớn, vùng đối tượng
Stage 4	2 blocks	512	Học đặc trưng mức cao (cấu trúc vật thể)

Lý do số lượng filter tăng dần là vì khi đi sâu vào mạng, kích thước không gian (chiều rộng/chiều cao) giảm nhưng mức độ trừu tượng của thông tin tăng - để biểu diễn những khái niệm trừu tượng hơn cần nhiều “kênh” (filters) hơn.

2.4.3.3. Bước tạo vector đặc trưng 512 chiều - global average pooling

Sau khi bức ảnh đi qua 4 stage của ResNet-18, mạng tạo ra một tensor có kích thước 512 kênh, mỗi kênh là một bản đồ đặc trưng kích thước 7×7 .

ResNet áp dụng lớp Global Average Pooling (GAP) để đơn giản hóa quá trình tạo đặc trưng.



Hình 2.10: Lớp Global Average Pooling

Cách hoạt động của GAP:

- Mỗi kênh đầu ra của ResNet ở stage cuối có kích thước 7×7 .
- GAP lấy giá trị trung bình của toàn bộ vùng 7×7 này.
- Nhờ đó, mỗi kênh 7×7 được nén thành một giá trị duy nhất.

Khi toàn bộ 512 kênh được xử lý theo cách này, mô hình thu được vector 512 chiều. Đây chính là dạng đặc trưng gọn, ổn định và giàu thông tin mà các hệ thống như tìm kiếm ảnh hoặc phân cụm thường sử dụng.

Lợi ích của GAP:

- Giảm mạnh số lượng tham số so với việc dùng nhiều lớp FC.
- Hạn chế overfitting.
- Giúp mô hình nhẹ hơn và dễ huấn luyện hơn.
- Vẫn giữ lại các đặc trưng quan trọng mà mạng học được ở các stage sâu cuối cùng.

2.4.3.4. Lớp Fully connected và Softmax

Lớp fully connected (FC) nằm ở cuối ResNet-18 có nhiệm vụ “dịch” vector 512 chiều thành các điểm số (logits) tương ứng với mỗi lớp phân loại (ví dụ 1000 lớp của ImageNet): về bản chất nó nhân vector 512 với một ma trận trọng số kích thước 1000×512 rồi cộng bias, sau đó đưa qua softmax để dự đoán nhãn. Vì vậy lớp FC chứa rất nhiều tham số và chuyên hóa cho nhiệm vụ phân loại cụ thể mà mô hình được huấn luyện (ví dụ ImageNet).

Khi trích xuất các đặc trưng chung dùng cho tìm kiếm ảnh, ta bỏ lớp FC vì:

- Lớp FC là “cửa ra” dành cho phân loại, nó biến feature thành nhãn đặc thù, làm mất tính tổng quát của vector.
- FC tăng mạnh số tham số và không cần thiết cho việc so sánh vector.
- Giữ lại vector 512 chiều sau Global Average Pooling cho phép ta có đại diện gọn, chung và dễ so sánh.

Bên cạnh đó, softmax là một hàm toán học chuyển một vector số bất kỳ thành một bộ xác suất có tổng bằng 1, khi dùng để trích xuất đặc trưng, hàm này cũng bị bỏ vì ta không cần dự đoán nhãn.

2.5. Độ đo tương đồng Cosine Similarity

2.5.1. Khái niệm

Trong hệ thống truy xuất ảnh tương đồng, sau khi đi qua mô hình ResNet18, mỗi ảnh được biểu diễn dưới dạng một vector đặc trưng có 512 chiều. Để xác định mức độ giống nhau giữa hai ảnh, cần sử dụng một độ đo tương đồng giữa các vector đặc trưng này.

Cosine Similarity là độ đo đánh giá mức độ tương đồng giữa hai vector dựa trên góc tạo bởi chúng trong không gian nhiều chiều. Thay vì so sánh độ lớn của vector, Cosine Similarity tập trung vào sự tương đồng về hướng của các vector.

Độ đo này được sử dụng rộng rãi trong các bài toán truy xuất thông tin, tìm kiếm ảnh tương tự và xử lý ngôn ngữ tự nhiên do khả năng đánh giá tốt mối quan hệ giữa các vector đặc trưng có số chiều lớn.

2.5.2. Công thức

Cosine Similarity giữa hai vector x và y được xác định theo công thức:

$$\text{Cosine Similarity}(x, y) = \frac{x \cdot y}{\|x\| \|y\|}$$

Trong đó:

- x và y là hai vector đặc trưng.
- $x \cdot y$ là tích vô hướng của hai vector.
- $\|x\|$ và $\|y\|$ là độ dài (chuẩn Euclid) của các vector.

Trong đề tài, các vector đặc trưng được chuẩn hóa L2 (L2 Normalization) trước khi tìm kiếm. Khi đó:

$$\|x\| = \|y\| = 1$$

nên công thức trở thành:

$$\text{Cosine Similarity}(x, y) = x \cdot y$$

Nhờ vậy, thư viện FAISS có thể sử dụng chỉ mục IndexFlatIP (Inner Product) để thực hiện tìm kiếm hiệu quả mà vẫn đảm bảo tương đương với độ đo cosine.

2.5.3. Ý nghĩa

Giá trị Cosine Similarity nằm trong khoảng từ -1 đến 1 :

Bảng 2.2: Ý nghĩa giá trị Cosine trong khoảng $(-1, 1)$

Giá trị	Ý nghĩa
Gần 1	Hai vector rất giống nhau, ảnh có nội dung tương đồng cao
Gần 0	Hai vector ít liên quan
Gần -1	Hai vector có hướng ngược nhau hoàn toàn

Đối với các vector đặc trưng được sinh ra từ ResNet18 và đã chuẩn hóa L2, giá trị cosine thường nằm trong khoảng từ 0 đến 1. Giá trị càng lớn cho thấy mức độ tương đồng giữa hai ảnh càng cao.

Trong hệ thống, Cosine Similarity được sử dụng để so sánh vector đặc trưng của ảnh truy vấn với các vector trong cơ sở dữ liệu, từ đó xác định và trả về các ảnh có mức độ tương đồng cao nhất.

2.6. Thư viện FAISS

2.6.1. Giới thiệu FAISS

FAISS (Facebook AI Similarity Search) là thư viện mã nguồn mở do nhóm nghiên cứu AI của Meta phát triển, chuyên dùng để tìm kiếm điểm gần nhất và tìm kiếm tương đồng trên các tập vector đặc trưng có số chiều lớn. FAISS được xây dựng bằng C++ và có giao diện Python, tối ưu cho cả CPU và GPU, có thể xử lý hiệu quả các tập dữ liệu chứa hàng triệu đến hàng tỷ vector.

Trong lĩnh vực thị giác máy tính, FAISS được sử dụng rộng rãi trong các bài toán truy xuất ảnh theo nội dung (Content-Based Image Retrieval - CBIR), nhận dạng khuôn mặt, tìm kiếm đa phương tiện và các hệ thống khai thác dữ liệu dựa trên vector đặc trưng. Trong đề tài này, FAISS được sử dụng để tìm kiếm các ảnh có mức độ tương đồng cao với ảnh truy vấn dựa trên vector đặc trưng được trích xuất từ mô hình ResNet18.

2.6.2. Tìm kiếm Exact và Approximate

FAISS hỗ trợ hai nhóm phương pháp tìm kiếm chính:

Exact Search

Exact Search (tìm kiếm chính xác) là phương pháp so sánh trực tiếp vector truy vấn với toàn bộ vector trong cơ sở dữ liệu để tìm ra các kết quả gần nhất.

Ưu điểm:

- Độ chính xác tuyệt đối;
- Kết quả truy xuất giống với phương pháp vét cạn (Brute Force).

Nhược điểm:

- Thời gian xử lý tăng khi số lượng vector rất lớn.

Approximate Search

Approximate Search (tìm kiếm gần đúng) sử dụng các kỹ thuật phân cụm hoặc nén dữ liệu nhằm giảm số lượng vector cần so sánh.

Ưu điểm:

- Tốc độ tìm kiếm rất cao;
- Phù hợp với các hệ thống quy mô lớn.

Nhược điểm:

- Có thể bỏ sót một số kết quả tối ưu;
- Độ chính xác thấp hơn Exact Search.

Trong phạm vi đề tài, nhóm lựa chọn phương pháp Exact Search để đảm bảo độ chính xác của kết quả truy xuất.

2.6.3. IndexFlatIP

FAISS tổ chức dữ liệu thành các cấu trúc gọi là Index, đây là thành phần lưu trữ toàn bộ vector đặc trưng và cung cấp các hàm tìm kiếm. Một số loại Index:

- **IndexFlatL2**: tìm kiếm chính xác dựa trên khoảng cách Euclid (L2). Duyệt toàn bộ vector trong kho để tìm ra K vector gần nhất, độ chính xác tuyệt đối nhưng tốc độ giảm khi dữ liệu lớn.
- **IndexFlatIP**: tìm kiếm chính xác dựa trên tích vô hướng (Inner Product). Khi các vector đã được chuẩn hóa L2 (đưa về độ dài 1), tích vô hướng giữa hai vector chính là độ tương đồng cosine. Đây là loại index được sử dụng trong dự án Image-Retrieval-System của nhóm.
- **IndexIVFFlat**: chia không gian vector thành nhiều cụm (cluster) bằng thuật toán phân cụm (k-means), khi tìm kiếm chỉ duyệt trong một số cụm gần nhất thay vì toàn bộ dữ liệu - giúp tăng tốc độ nhưng độ chính xác là gần đúng.
- **IndexPQ (Product Quantization)**: nén vector thành các mã ngắn để giảm dung lượng bộ nhớ, phù hợp khi dữ liệu cực lớn (hàng trăm triệu đến hàng tỷ vector).
- **IndexHNSW**: dựa trên đồ thị phân lớp, cho tốc độ tìm kiếm nhanh với độ chính xác cao, thường dùng trong các hệ thống production quy mô lớn.

Trong dự án Image-Retrieval-System, FAISS được cấu hình theo phương án IndexFlatIP kết hợp với việc chuẩn hóa L2 toàn bộ vector đặc trưng trước khi đưa vào index. Lý do lựa chọn này:

- Độ chính xác tuyệt đối: IndexFlatIP là tìm kiếm chính xác, không nén hay xấp xỉ vector, nên kết quả top-K trả về luôn đúng theo độ tương đồng cosine, không có sai số như các phương pháp xấp xỉ (IVF, PQ, HNSW).
- Tốc độ vẫn rất nhanh ở quy mô vừa: với 60.000 vector 512 chiều (bộ CIFAR-10), thời gian truy vấn trung bình chỉ khoảng 1-3 mili-giây cho mỗi lượt tìm kiếm.
- Cosine similarity phù hợp với học sâu: vector đặc trưng từ ResNet-18 hay CLIP thường mang ý nghĩa về hướng nhiều hơn độ lớn, nên đo độ tương đồng theo góc giữa hai vector (cosine) cho kết quả ngữ nghĩa tốt hơn so với khoảng cách Euclid thông thường.

- Khả năng mở rộng: khi dữ liệu tăng lên hàng triệu/tỷ ảnh, có thể chuyển sang các index gần đúng như IVF, PQ hoặc HNSW để đánh đổi một phần độ chính xác lấy tốc độ.

2.6.4. Chuẩn hóa L2

Trước khi xây dựng chỉ mục FAISS, các vector đặc trưng được thực hiện chuẩn hóa L2 nhằm đưa độ dài của tất cả các vector về giá trị bằng 1.

Đối với vector đặc trưng ($x = (x_1, x_2, \dots, x_n)$), chuẩn L2 xác định như sau:

$$|x| = \sqrt{x_1^2 + x_2^2 + \dots + x_n^2}$$

Vector sau khi chuẩn hóa được tính bởi:

$$x' = \frac{x}{|x|}$$

Khi đó:

$$|x'| = 1$$

Việc chuẩn hóa L2 giúp loại bỏ ảnh hưởng của độ lớn vector và chỉ giữ lại thông tin về hướng của vector trong không gian đặc trưng. Đây là bước tiền xử lý quan trọng đối với các bài toán sử dụng độ đo Cosine Similarity.

Sau khi chuẩn hóa L2, công thức Cosine Similarity được rút gọn thành:

$$\text{Cosine Similarity}(x, y) = x \cdot y$$

Do đó, phép tính tích vô hướng (Inner Product) được sử dụng trong IndexFlatIP trở nên tương đương với độ đo Cosine Similarity.

2.6.5. Quy trình xây dựng và sử dụng index

- Trích xuất đặc trưng: mỗi ảnh trong kho dữ liệu được đưa qua mạng ResNet-18 để thu được vector đặc trưng (ví dụ 512 chiều).
- Chuẩn hóa vector: toàn bộ vector được chuẩn hóa L2 (đưa về độ dài đơn vị) để tích vô hướng tương đương cosine similarity.
- Xây dựng Index: các vector đã chuẩn hóa được nạp vào IndexFlatIP của FAISS, tạo thành kho index lưu trong bộ nhớ (hoặc lưu file để tái sử dụng).
- Truy vấn: khi người dùng đưa vào một ảnh hoặc một mô tả văn bản, hệ thống trích đặc trưng của truy vấn theo cùng quy trình, sau đó dùng hàm search() của FAISS để tìm ra K vector gần nhất trong index, trả về chỉ số (index) và điểm tương đồng (score) tương ứng.
- Ánh xạ kết quả: chỉ số vector trả về được ánh xạ ngược lại với ảnh gốc và nhãn lớp để hiển thị kết quả cho người dùng.

2.6.6. Ưu điểm của FAISS

FAISS sở hữu nhiều đặc điểm phù hợp với các hệ thống truy xuất ảnh dựa trên vector đặc trưng học sâu.

- Hỗ trợ xử lý hiệu quả dữ liệu có số chiều lớn.
- Cung cấp nhiều cấu trúc chỉ mục phục vụ các yêu cầu truy xuất khác nhau.
- Hỗ trợ cả tìm kiếm chính xác và tìm kiếm gần đúng.
- Có khả năng mở rộng đối với các tập dữ liệu quy mô lớn.
- Tương thích tốt với các vector đặc trưng sinh ra từ các mô hình học sâu.
- Hỗ trợ tăng tốc bằng GPU đối với các ứng dụng yêu cầu hiệu năng cao.

Trong đề tài này, FAISS được lựa chọn làm công cụ truy xuất chính nhờ khả năng thực hiện tìm kiếm theo độ tương đồng cosine một cách hiệu quả, đồng thời vẫn duy trì độ chính xác tương đương với phương pháp KNN truyền thống.

2.7. CLIP (Contrastive Language–Image Pre-training)

Mô hình CLIP (Contrastive Language-Image Pre-training) do OpenAI công bố đã đánh dấu một bước ngoặt trong lĩnh vực trí tuệ nhân tạo đa phương thức. Thay vì sử dụng phương pháp học có giám sát trên một bộ nhãn cố định, CLIP học các biểu diễn thị giác thông qua giám sát từ ngôn ngữ tự nhiên trên 400 triệu cặp dữ liệu ảnh và văn bản.

2.7.1. Tổng quan

Trong nhiều thập kỷ, các hệ thống thị giác máy tính chủ yếu dựa vào phương pháp học có giám sát trên các bộ dữ liệu được gán nhãn thủ công như ImageNet. Mạng nơ-ron tích chập (CNN) sẽ được huấn luyện để phân loại hình ảnh vào một tập hợp danh mục giới hạn. Mặc dù đạt độ chính xác cao trên các tập kiểm thử, cách tiếp cận này vấp phải hai rào cản lớn:

- Thiếu tính khái quát (Generalization): Mô hình không thể nhận diện các đối tượng nằm ngoài danh sách nhãn đã định trước mà không được huấn luyện lại từ đầu.
- Khoảng trống độ mạnh (Robustness Gap): Mô hình thường "học vẹt" phân phối của dữ liệu huấn luyện, dẫn đến suy giảm hiệu năng nghiêm trọng khi áp dụng vào môi trường thực tế.

Để giải quyết triệt để rào cản này, OpenAI phát triển CLIP với mục tiêu loại bỏ danh mục cố định. CLIP học khái niệm thị giác trực tiếp từ ngôn ngữ tự nhiên thông qua 400 triệu cặp (hình ảnh, văn bản) được thu thập trên Internet, mang lại khả năng hiểu biết thế giới mở và linh hoạt

2.7.2. Kiến trúc cốt lõi của hệ thống CLIP

Động lực chính của CLIP là tạo ra một cơ chế cho phép chuyển đổi và đồng bộ hóa hai dạng dữ liệu hoàn toàn khác biệt (hình ảnh dạng ma trận pixel và văn bản dạng chuỗi ký tự) vào cùng một hệ trục tọa độ toán học. Để làm được điều này, CLIP sử dụng một kiến trúc tháp đôi (dual-encoder architecture), bao gồm một bộ mã hóa hình ảnh (Image Encoder) và một bộ mã hóa văn bản (Text Encoder) hoạt động hoàn toàn độc lập với nhau, sau đó kết nối thông qua một lớp chiếu tuyến tính (linear projection).

Điểm đặc biệt của quy trình này là cả hai bộ mã hóa đều được huấn luyện hoàn toàn từ đầu (from scratch), không hề khởi tạo từ các trọng số đã được huấn luyện trước trên ImageNet cho ảnh hay trọng số pre-trained cho văn bản, đảm bảo mô hình không kế thừa các định kiến phân phối dữ liệu từ quá khứ.

Do hình ảnh (ma trận pixel) và văn bản (chuỗi ký tự) là hai dạng dữ liệu không đồng nhất, CLIP xử lý chúng thông qua một thiết kế "tháp đôi" hoạt động song song. Cả hai bộ mã hóa đều được huấn luyện từ đầu (from scratch).

- Bộ mã hóa Hình ảnh (Image Encoder): Dựa trên kiến trúc ResNet-50 (tùy biến) hoặc Vision Transformer (ViT). Thực nghiệm chứng minh ViT mang lại hiệu năng tính toán vượt trội, giúp mô hình huấn luyện nhanh hơn gấp 3 lần so với ResNet truyền thống.
- Bộ mã hóa Văn bản (Text Encoder): Sử dụng kiến trúc Transformer tiêu chuẩn. Đầu vào được giới hạn ở mức 77 tokens (phù hợp với các mô tả ngắn trên mạng) và được kẹp giữa hai token đặc biệt là (bắt đầu) và (kết thúc).

2.7.3. Cơ sở toán học và hàm mất mát (Contrastive Learning)

Thay vì sử dụng phương pháp tạo sinh (generative) đắt đỏ để dự đoán từng từ, CLIP sử dụng cơ chế Học Đối nghịch (Contrastive Learning) thông qua bài toán phân loại N - cặp đa lớp. Mục tiêu cốt lõi của CLIP là chiếu cả dữ liệu ảnh và chữ lên cùng một không gian nhúng chung (joint embedding space). Quy trình toán học trong mỗi lô dữ liệu (batch) kích thước N diễn ra như sau:

Bước 1: Trích xuất và Chiếu không gian

Bộ mã hóa ảnh sinh ra véc-tơ đặc trưng ảnh I_f , bộ mã hóa chữ sinh ra véc-tơ đặc trưng chữ T_f . Chúng được nhân với các ma trận trọng số tương ứng W_i và W_t , sau đó trải qua chuẩn hóa L2 (L2 Normalization) để tạo thành các véc-tơ đơn vị:

$$I_e = \frac{I_f W_i}{\|I_f W_i\|_2}$$

$$T_e = \frac{T_f W_t}{\|T_f W_t\|_2}$$

Bước 2: Tính ma trận tương đồng (Logits)

Tính toán ma trận độ tương đồng Cosine kích thước $N \times N$, khuếch đại bằng tham số nhiệt độ τ có thể học được. Đối với ảnh thứ i và văn bản thứ j :

$$S_{i,j} = \langle I_e, T_e \rangle \cdot \exp(\tau)$$

Bước 3: Tối ưu hóa hàm mất mát đối xứng

Chỉ có đường chéo chính ($i = j$) là các cặp ghép đúng. Mô hình tối ưu hóa đồng thời theo hai chiều:

- Mất mát chiều ảnh tìm chữ (tính theo hàng):

$$L_{i \rightarrow t} = -\frac{1}{N} \sum_{i=1}^N \log \frac{\exp(S_{i,i})}{\sum_{j=1}^N \exp(S_{i,j})}$$

- Mất mát chiều chữ-tìm-ảnh (tính theo cột):

$$L_{t \rightarrow i} = -\frac{1}{N} \sum_{i=1}^N \log \frac{\exp(S_{i,i})}{\sum_{j=1}^N \exp(S_{j,i})}$$

- Hàm mất mát tổng thể (L_{CLIP}) là trung bình cộng của cả hai

$$L_{CLIP} = \frac{1}{2} (L_{i \rightarrow t} + L_{t \rightarrow i})$$

Thuật toán cực đại hóa các phần tử trên đường chéo và cực tiểu hóa các phần tử nhiễu, giúp tăng hiệu năng tính toán lên 4-10 lần so với mô hình cũ.

2.7.4. Phân loại Zero-Shot và kỹ thuật điều hướng (Prompt Engineering)

Thuật ngữ "Zero-shot learning" đề cập đến khả năng của một hệ thống có thể nhận dạng, phân loại hoặc tương tác với một tập dữ liệu hoặc khái niệm mới hoàn toàn, mà không cần nhận bất kỳ một dữ liệu mẫu huấn luyện nào liên quan đến nhãn đó trong pha tinh chỉnh (fine-tuning). CLIP đã mở ra một kỷ nguyên nơi khái niệm zero-shot trở thành chuẩn mực thực tế cho các nền tảng thị giác máy tính.

2.7.4.1. Thay đổi hệ thức phân loại Zero-Shot

Để sử dụng CLIP như một công cụ phân loại ảnh zero-shot, quá trình suy luận diễn ra hoàn toàn khác biệt so với các phương pháp cũ. Trình tự thực thi như sau: người dùng chuẩn bị một danh sách các "nhãn" mà họ muốn hệ thống nhận diện. Thay vì giữ nguyên nhãn dưới dạng chỉ mục số nguyên vô hồn (ví dụ: class

1, class 2), họ sẽ biến nó thành ngôn ngữ tự nhiên, chẳng hạn như "con ngựa vằn", "chiếc xe máy". Mỗi văn bản này sẽ được bộ mã hóa văn bản chuyển thành một véc-tơ đa chiều biểu thị ngữ nghĩa, tạo thành một bộ phân loại tuyến tính tổng hợp. Khi một hình ảnh mới xuất hiện, bộ mã hóa hình ảnh của CLIP trích xuất véc-tơ thị giác của nó, sau đó hệ thống thực hiện tính toán độ tương đồng giữa véc-tơ ảnh và tất cả các véc-tơ nhãn. Lớp có điểm tương đồng cao nhất sẽ được xuất ra dưới dạng kết quả dự đoán cuối cùng.

Cơ chế này mang đến sự linh hoạt tối thượng. Trái ngược với các mô hình trước đây cần thu thập hàng nghìn ảnh cho một nhãn mới và tiến hành tái cấu trúc, tái huấn luyện toàn bộ mạng phân loại, CLIP cho phép các nhà phát triển cập nhật và mở rộng vốn từ vựng nhận diện ngay trong thời gian thực chỉ bằng cách thay đổi đoạn văn bản truy vấn. Khi được kiểm tra trên hơn 30 bộ dữ liệu máy tính khác nhau (bao gồm nhận diện hành động trong video, vị trí địa lý, nhận diện ký tự quang học OCR, và phân loại vật thể chi tiết), CLIP chứng tỏ hiệu năng xuất sắc, thường ngang bằng hoặc đánh bại các mô hình có giám sát hoàn toàn. Đáng chú ý, tính năng học OCR vốn là một hành vi không hề xuất hiện trong các mô hình đào tạo theo chuẩn ImageNet thông thường.

2.7.4.2. Đóng lại khoảng trống độ mạnh

Trong lịch sử nghiên cứu AI, các hệ thống học sâu thường mắc phải hội chứng "gian lận độ mạnh" (robustness gap). Chúng tối ưu hóa quá mức và "học vẹt" các phân phối dữ liệu thuộc một thước đo chuẩn (benchmark) nhất định như ImageNet. Khi đem mô hình đó thử nghiệm trên thế giới thực hoặc các tập dữ liệu có sự biến dạng nhẹ về nhiễu, phong cách vẽ, độ phân giải, hiệu suất của chúng sụp đổ nghiêm trọng do mô hình chỉ nắm bắt được các tương quan giả mạo thay vì bản chất của vật thể.

Bởi vì bản chất của CLIP là một bộ trích xuất đặc trưng chưa bao giờ tiếp xúc trực tiếp hay tối ưu hóa chuyên biệt cho bất kỳ bộ dữ liệu đánh giá nào, nó hoàn toàn miễn nhiễm với hiện tượng "học vẹt" này. Các thử nghiệm chỉ ra rằng, cấu trúc zero-shot của CLIP làm giảm khoảng trống độ mạnh lên tới 75% trong khi vẫn giữ nguyên hiệu năng tổng thể trên tập ImageNet so với nguyên bản ResNet-50 có giám sát (không sử dụng 1,28 triệu mẫu được dán nhãn của ImageNet). Khi một bộ phân loại tuyến tính được huấn luyện đề lên các đặc trưng của CLIP, độ chính xác có thể tăng 10% trên ImageNet, nhưng khả năng kháng nhiễu trên 7 bộ dữ liệu kiểm tra độ mạnh lại không hề tăng lên, chứng minh rằng sự tối ưu hóa trực tiếp luôn kéo theo sự hy sinh về khả năng tổng quát hóa.

2.7.4.3. Sự ra đời của kỹ sư dấu nhắc

Một phát hiện quan trọng trong hành trình nghiên cứu và tối ưu hóa hệ thống là CLIP vô cùng nhạy cảm với cách các truy vấn văn bản được diễn đạt. Vì mô

hình được học từ các câu miêu tả tự nhiên thay vì những từ vựng đơn độc trong từ điển, việc cung cấp một từ đơn lẻ (như "chó") không mang lại hiệu năng tối ưu bằng việc xây dựng một ngữ cảnh giả lập bằng câu hoàn chỉnh⁸.

Nhóm nghiên cứu nhận thấy rằng việc bọc nhãn phân loại bằng các khuôn mẫu (templates) văn bản như "Một bức ảnh của một {nhãn}" (A photo of a {label}) có thể cải thiện đáng kể độ chính xác của quá trình suy luận zero-shot. Đối với những bộ dữ liệu mang tính chuyên biệt cao, việc mở rộng khuôn mẫu có thể mang lại hiệu suất tốt hơn nữa. Ví dụ: đối với phân loại vật thể nuôi trong nhà, việc sử dụng khuôn mẫu "Một bức ảnh của một {nhãn}, một loại thú cưng" cung cấp một mỏ neo ngữ nghĩa, giúp định hướng không gian nhúng của bộ mã hóa văn bản về phía một cụm (cluster) không gian liên quan đến vật nuôi, từ đó hệ thống phân loại chính xác hơn so với chỉ dùng từ khóa trần trụi. Hành vi này đã khai sinh ra một phân ngành hoàn toàn mới: Prompt Engineering trong lĩnh vực thị giác máy tính. Hơn nữa, việc sử dụng phương pháp "Bag-of-Words" để kết hợp với kỹ thuật cân bằng từ (word balancing) hoặc tạo thành bộ quần thể các prompt (ensembling) đã được thực nghiệm là một công cụ hiệu chỉnh zero-shot cực kỳ mạnh mẽ.

2.7.5. Ứng dụng trong AI sinh tạo (Generative AI)

Dù bản chất là một mô hình phân loại đối nghịch và không thể tự sinh ra hình ảnh từ văn bản, một trong những hệ quả cách mạng nhất của CLIP lại là vai trò tối quan trọng của nó trong các công cụ trí tuệ nhân tạo sinh ảnh (Text-to-Image Generative AI) như Stable Diffusion, DALL-E, hay Midjourney. CLIP đóng vai trò như cầu nối lý tưởng giữa ý định của người dùng và quá trình tạo ảnh.

Trong kiến trúc của Stable Diffusion (SD), một mô hình khuếch tán tiềm ẩn (latent diffusion model), quá trình khử nhiễu (denoising) cần phải được hướng dẫn bởi ngữ nghĩa của văn bản (conditional generation). Cấu trúc của Stable Diffusion bao gồm nhiều thành phần phối hợp: một mạng Autoencoder VAE (thường là VQ-GAN) làm nhiệm vụ nén hình ảnh từ độ phân giải 512x512 xuống một không gian tiềm ẩn kích thước 64x64, và một mạng U-Net đóng vai trò loại bỏ nhiễu từng bước một. Tuy nhiên, bản thân Stable Diffusion không tự học cách hiểu văn bản; nó vay mượn trực tiếp bộ mã hóa văn bản đã đóng băng (frozen text-encoder) của CLIP, cụ thể là biến thể clip-vit-large-patch14, để làm nhiệm vụ diễn dịch câu lệnh.

Khi người dùng nhập một câu lệnh (prompt), CLIPTextModel sẽ chuyển đổi các token văn bản thành một ma trận biểu diễn tiềm ẩn ngưng tụ toàn bộ cấu trúc ý nghĩa của câu nói. Vector này sau đó được bơm vào kiến trúc U-Net của mạng khuếch tán thông qua một cơ chế chú ý chéo (Cross-Attention) ở từng bước giải nhiễu, định hướng thuật toán hình thành các chi tiết phù hợp với mô tả. Sự hiểu

biết nguyên thâm của CLIP về liên kết ngôn ngữ-hình ảnh chính là "hệ điều hành ngữ nghĩa" cho phép Stable Diffusion có thể vẽ ra mọi thứ, từ một thực thể cụ thể đến một phong cách hội họa siêu thực, ứng dụng mạnh mẽ trong sáng tạo nghệ thuật, thương mại điện tử và quảng cáo.

Gần đây, các nghiên cứu tiên bộ đã chỉ ra những hướng đi mới nhằm khai thác triệt để sức mạnh của CLIP trong hệ sinh thái khuếch tán. Mặc dù việc đóng băng bộ mã hóa văn bản CLIP hoạt động tốt, một số nghiên cứu chứng minh rằng nếu áp dụng việc tinh chỉnh (fine-tuning) bộ mã hóa văn bản thông qua cơ chế phản hồi phần thưởng (TextCrafter), chất lượng thẩm mỹ và mức độ bám sát câu lệnh (text-image alignment) của ảnh sinh ra tăng lên đáng kể. Cách tiếp cận này giúp cải thiện đầu ra mà không cần phải chuyển sang dùng các mô hình ngôn ngữ lớn như T5 vốn đòi hỏi nguồn lực điện toán cực cao. Hơn nữa, thông qua một ma trận chiếu tuyến tính được tính toán khép kín, bản thân bộ mã hóa hình ảnh của CLIP có thể chuyển đổi trực tiếp hình ảnh thành các prompt, phục vụ cho các tác vụ chỉnh sửa ảnh (image editing) và biến thể ảnh (image variation).

2.7.6. Triển khai kỹ thuật thực tiễn, hệ sinh thái API và vi dịch vụ

Trong framework của Hugging Face transformers, kiến trúc CLIP được trừu tượng hóa và đóng gói trong một quy trình duy nhất thông qua bộ đôi API là CLIPProcessor và CLIPModel. Đối tượng CLIPProcessor làm nhiệm vụ gộp toàn bộ quy trình tiền xử lý đa phương tiện. Nó tích hợp CLIPImageProcessor (chịu trách nhiệm chia tỷ lệ, cắt vùng trung tâm, và chuẩn hóa ma trận dữ liệu ảnh) cùng với CLIPTokenizer (cắt chữ thành các đơn vị sub-word, chèn token đặc biệt, và đệm/cắt độ dài tối đa) vào một hàm thực thi duy nhất. Khi đầu vào hình ảnh (từ đối tượng PIL.Image hoặc cấu trúc mảng numpy) cùng với chuỗi nhãn văn bản được nạp vào, hàm sẽ tự động trả về định dạng tensor (ví dụ pt cho PyTorch) sẵn sàng cho ma trận tính toán.

Sau khi dữ liệu được chuẩn bị, gọi đối tượng model(inputs) sẽ trực tiếp sinh ra hai tensor kết quả là logits_per_image và logits_per_text. Bằng cách tính toán softmax dọc theo chiều tương ứng của các tensor này, hệ thống xuất ra ma trận phân phối xác suất phán đoán của hình ảnh với các mô tả văn bản, trả về độ tự tin của mô hình đối với từng nhãn cụ thể. Gần đây, việc tối hợp các hàm nhúng tối ưu phần cứng như Flash Attention 2 (FA2) và SDPA (Scaled Dot-Product Attention) vào kiến trúc PyTorch cốt lõi đã giúp tăng tốc độ suy luận của mô hình ngôn ngữ CLIPTextModel lên từ 1,14 lần đến 1,26 lần, và cải thiện tốc độ luồng dữ liệu của toàn bộ CLIPModel trung bình khoảng 1,2 lần ở các khối lượng lô dữ liệu lớn.

Đối với thư viện chuẩn OpenAI CLIP trên GitHub, các lập trình viên có thể truy xuất trực tiếp các trọng số thông qua lệnh clip.available_models() và tải hệ

thống vào RAM/VRAM bằng cấu trúc lệnh `clip.load(name, device, jit=False)`. Tương tự, OpenCLIP cung cấp hàm `open_clip.create_model_from_pretrained` kết nối trực tiếp với trung tâm lưu trữ (Hub) để nạp các mô hình tỷ tham số chỉ bằng một dòng lệnh.

Bảng 2.3: Các thư viện và tính năng sử dụng

Thư viện/Nền tảng	Tính năng cốt lõi và cơ chế Triển khai	Phương thức trích xuất/định dạng đầu ra
OpenAI CLIP gốc	Tải mô hình ResNet/ViT, token hóa văn bản cơ bản thông qua hàm <code>clip.tokenize()</code> .	Gọi <code>model.encode_image()</code> và <code>model.encode_text()</code> để lấy tensor đặc trưng 1D.
Hugging Face Transformers	Đóng gói toàn trình Image và Text với <code>CLIPProcessor</code> . Tích hợp hoàn toàn vào hệ sinh thái Pipeline Inference.	Đầu ra hỗ trợ cả PyTorch (pt) và NumPy (np). Sinh <code>logits_per_image</code> tự động.
OpenCLIP (LAION)	Hỗ trợ tải mô hình mã nguồn mở thế hệ mới (ví dụ mô hình LAION-5B, SigLIP). Tích hợp huấn luyện phân tán đa GPU.	Hàm <code>create_model_from_pretrained()</code> , tối ưu hóa trực tiếp tính năng trích xuất zero-shot.

CHƯƠNG 3: TRIỂN KHAI, ĐÁNH GIÁ HỆ THỐNG

3.1. Môi trường thực nghiệm

Để triển khai và đánh giá hệ thống truy xuất ảnh tương đồng, nhóm sử dụng môi trường phần cứng và phần mềm được trình bày trong Bảng 3.1.

Bảng 3.1: Môi trường thực nghiệm

Thành phần	Thông tin
Hệ điều hành	Windows 11 64-bit
Ngôn ngữ lập trình	Python 3.12
Framework học sâu	PyTorch
Thư viện xử lý ảnh	TorchVision
Thư viện tính toán	NumPy
Thư viện học máy	Scikit-Learn
Thư viện tìm kiếm vector	FAISS
Framework Web	Flask
Công cụ giao diện	HTML, CSS, JavaScript
Bộ dữ liệu	CIFAR-10
Môi trường phát triển	Visual Studio Code

3.2. Bộ dữ liệu thực nghiệm

3.2.1. Giới thiệu bộ dữ liệu CIFAR-10

Trong nghiên cứu này, nhóm sử dụng bộ dữ liệu CIFAR-10 để xây dựng và đánh giá hệ thống truy xuất ảnh tương đồng. CIFAR-10 là một trong những bộ dữ liệu chuẩn được sử dụng rộng rãi trong lĩnh vực thị giác máy tính và học sâu.

Bộ dữ liệu được phát triển bởi Alex Krizhevsky cùng các cộng sự tại University of Toronto. CIFAR-10 bao gồm 60.000 ảnh màu, kích thước 32×32 pixel, được chia thành 10 lớp đối tượng khác nhau, mỗi lớp chứa 6.000 ảnh.

Các lớp đối tượng trong CIFAR-10 gồm:

STT	Lớp dữ liệu
1	Airplane
2	Automobile
3	Bird
4	Cat

5	Deer
6	Dog
7	Frog
8	Horse
9	Ship
10	Truck

Bộ dữ liệu được chia thành:

- 50.000 ảnh huấn luyện (Training Set).
- 10.000 ảnh kiểm thử (Test Set).

Trong đề tài, toàn bộ 60.000 ảnh được sử dụng để xây dựng cơ sở dữ liệu ảnh phục vụ quá trình truy xuất và đánh giá hệ thống.

3.2.2. Tiền xử lý dữ liệu

Trước khi đưa vào mô hình ResNet18, dữ liệu ảnh được tiền xử lý nhằm đảm bảo tính tương thích với mô hình học sâu đã được huấn luyện trên ImageNet.

Các bước tiền xử lý được thực hiện gồm:

Bước 1. Thay đổi kích thước ảnh

Ảnh CIFAR-10 có kích thước gốc 32×32 pixel, trong khi ResNet18 yêu cầu đầu vào kích thước 224×224 pixel. Do đó, tất cả ảnh được nội suy và thay đổi kích thước về 224×224 pixel.

Bước 2. Chuyển đổi định dạng dữ liệu

Ảnh được chuyển từ định dạng RGB sang Tensor để phục vụ quá trình xử lý bằng thư viện PyTorch.

Bước 3. Chuẩn hóa dữ liệu

Giá trị điểm ảnh được chuẩn hóa theo bộ tham số của ImageNet nhằm đảm bảo sự tương thích với trọng số pretrained của mô hình ResNet18.

Quá trình chuẩn hóa được thực hiện theo công thức:

$$x' = \frac{x - \mu}{\sigma}$$

Trong đó:

- x : giá trị điểm ảnh ban đầu.
- μ : giá trị trung bình của tập ImageNet.
- σ : độ lệch chuẩn của tập ImageNet.
- x' : giá trị sau chuẩn hóa.

Kết quả sau tiền xử lý

Sau khi hoàn thành quá trình tiền xử lý:

- Tất cả ảnh được chuyển về kích thước 224×224 pixel.
- Dữ liệu được chuẩn hóa theo chuẩn ImageNet.
- Ảnh sẵn sàng đưa vào mô hình ResNet18 để trích xuất đặc trưng.

Kết quả là tập dữ liệu đầu vào đồng nhất, giúp nâng cao hiệu quả trích xuất đặc trưng và đảm bảo tính ổn định của hệ thống truy xuất ảnh tương đồng.

3.3. Quy trình xây dựng hệ thống

Bảng 3.2: Quy trình xây dựng hệ thống

Bước	Tên	Nội dung thực hiện trong đề tài
1	Thu thập dữ liệu	Sử dụng bộ CIFAR-10: 60.000 ảnh màu 32×32 , chia thành 10 lớp (máy bay, ô tô, chim, mèo, hươu, chó, ếch, ngựa, tàu, xe tải). Tải tự động qua torchvision.datasets.
2	Làm sạch & tiền xử lý	Chuyển ảnh về RGB, chuẩn hoá kích thước, normalize theo mean/std của ImageNet (khớp với model pretrained). Loại ảnh lỗi; lưu danh sách ảnh (image_list.txt) và nhãn (labels.npy).
3	Trích xuất đặc trưng (Feature Extraction)	Đưa toàn bộ 60.000 ảnh qua ResNet-18 $\rightarrow$ vector 512 chiều (features.npy); và qua CLIP ViT-B/32 $\rightarrow$ vector 512 chiều (features_clip.npy). Đây là bước biến ảnh thành "vector số" để so sánh.
4	Lựa chọn mô hình	Chọn ResNet-18 cho đặc trưng thị giác (nhẹ, 11.7M tham số, chạy CPU) và CLIP cho đặc trưng ngữ nghĩa (tìm bằng văn bản). Loại bỏ lớp phân loại cuối, chỉ lấy phần backbone trích đặc trưng.
5	Điều chỉnh tham số	L2-normalize vector (để inner product = cosine similarity); chọn K (số ảnh trả về, mặc định 10); cấu hình chỉ mục FAISS IndexFlatIP; xây từ điển dịch Việt $\rightarrow$ Anh cho truy vấn CLIP tiếng Việt.

6	Xây dựng chỉ mục & truy vấn	Nạp toàn bộ vector vào FAISS. Khi có truy vấn (ảnh hoặc văn bản) → vector hoá → so cosine với 60.000 vector → trả về top-K ảnh giống nhất.
7	Đánh giá & triển khai	Đo Precision@k, Recall@k, mAP; xây giao diện web (Flask) + chatbot; đóng gói bằng Docker và triển khai Hugging Face Spaces.

3.4. Triển khai hệ thống

3.4.1. Tổng quan kiến trúc hệ thống

Hệ thống được tổ chức theo mô hình Client-Server, gồm hai luồng xử lý chính:

Luồng offline (chạy một lần trước khi khởi động server): Toàn bộ 60.000 ảnh CIFAR-10 được đưa qua ResNet-18 và CLIP để trích xuất đặc trưng, kết quả lưu vào các file .npz. Bước này chỉ thực hiện một lần duy nhất, giúp server không phải chạy lại mô hình mỗi khi có truy vấn.

Luồng online (xử lý theo thời gian thực): Khi người dùng gửi ảnh hoặc mô tả văn bản, server trích xuất vector đặc trưng của ảnh/văn bản truy vấn, tìm kiếm trên FAISS index đã xây dựng sẵn và trả kết quả về giao diện web.

[Offline] CIFAR-10 → ResNet-18 → features.npz
→ CLIP → features_clip.npz

[Online] Upload ảnh → extract_feature() → FAISS search → Kết quả
Gõ mô tả → CLIP encode text → FAISS search → Kết quả

3.4.2. Trích xuất đặc trưng bằng ResNet18

Lựa chọn mô hình

Đề tài sử dụng mô hình ResNet18 được huấn luyện trước trên tập dữ liệu ImageNet thông qua phương pháp Transfer Learning.

Mô hình ResNet18 gồm 18 lớp học sâu với cơ chế Residual Learning giúp khắc phục hiện tượng suy giảm hiệu năng khi tăng độ sâu mạng.

Lý do lựa chọn:

- Kiến trúc gọn nhẹ.
- Tốc độ xử lý nhanh.
- Chất lượng đặc trưng tốt.
- Phù hợp với tài nguyên phần cứng của nhóm.

Cấu hình mô hình

Trong nghiên cứu này, lớp Fully Connected cuối cùng được loại bỏ và chỉ sử dụng phần Backbone của mạng để trích xuất đặc trưng, đầu ra của lớp Average Pooling được sử dụng làm vector biểu diễn ảnh.

```
Model = models.resnet18(weights=ResNet18_Weights.IMAGENET1K_V1)
model.fc = nn.Identity()
```

Khi đó đầu ra của mô hình là vector đặc trưng kích thước:

$$512 \times 1$$

Mỗi ảnh sẽ được biểu diễn bởi một vector gồm 512 giá trị số thực.

Quy trình trích xuất đặc trưng

Đối với mỗi ảnh:

1. Đọc ảnh từ bộ dữ liệu.
2. Resize về kích thước 224×224 .
3. Chuẩn hóa theo ImageNet.
4. Đưa qua ResNet18.
5. Thu được vector đặc trưng 512 chiều.

Sau khi hoàn thành bước trích xuất đặc trưng, toàn bộ vector được lưu vào các tệp dữ liệu.

– **features.npy**

Lưu ma trận đặc trưng: **60000×512**

Mỗi hàng tương ứng với một ảnh.

– **labels.npy**

Lưu nhãn lớp của từng ảnh.

– **image_list.txt**

Lưu ảnh dưới dạng Base64 để phục vụ hiển thị trên giao diện Web.

Việc lưu sẵn đặc trưng giúp hệ thống không phải chạy lại ResNet18 đối với toàn bộ tập dữ liệu mỗi lần khởi động.

3.4.3. Xây dựng bộ tìm kiếm FAISS

Chuẩn hóa vector

Trước khi xây dựng chỉ mục, tất cả vector đặc trưng được chuẩn hóa L2:

$$x' = \frac{x}{\|x\|}$$

Sau chuẩn hóa:

$$\|x'\| = 1$$

Điều này giúp phép nhân vô hướng tương đương với Cosine Similarity.

Xây dựng chỉ mục IndexFlatIP

Đề tài sử dụng chỉ mục: `faiss.IndexFlatIP()`

Đây là phương pháp tìm kiếm Exact Search được cung cấp bởi FAISS.

Các vector đặc trưng được thêm vào chỉ mục: `index.add(features)`

Tìm kiếm

Khi có ảnh truy vấn:

1. Trích xuất vector đặc trưng.
2. Chuẩn hóa L2.
3. Tìm kiếm Top-10 bằng FAISS.

`scores, indices = index.search(query, 10)`

3.4.4. Trích xuất đặc trưng bằng CLIP

Giới thiệu

Ngoài tìm kiếm bằng ảnh, hệ thống còn hỗ trợ tìm kiếm bằng mô tả văn bản tự nhiên (kể cả tiếng Việt) thông qua mô hình CLIP (Contrastive Language–Image Pretraining). CLIP được huấn luyện để nhúng cả ảnh lẫn văn bản vào **cùng một không gian vector** - cho phép so sánh trực tiếp giữa câu mô tả và ảnh trong kho.

Cấu hình

Đề tài sử dụng thư viện `open-clip-torch`. Toàn bộ 60.000 ảnh CIFAR-10 được mã hóa sẵn thành vector CLIP và lưu vào `features/features_clip.npy` (shape: 60.000×512). FAISS index riêng (`index_clip`) được xây dựng tương tự như phần ResNet-18.

Hỗ trợ tiếng Việt

CLIP gốc chỉ hiểu tiếng Anh. Module `clip_model.py` tích hợp bước dịch tự động Việt - Anh trước khi mã hóa văn bản, cho phép người dùng gõ mô tả tiếng Việt như "con mèo", "máy bay trên bầu trời" và hệ thống vẫn trả về kết quả đúng.

Tìm kiếm theo văn bản

Khi người dùng gõ mô tả:

1. Dịch văn bản Việt-Anh (nếu cần)
2. Mã hóa văn bản bằng CLIP text encoder → vector 512 chiều
3. L2-normalize vector văn bản
4. Tìm kiếm top-K trên `index_clip` bằng FAISS (giống tìm kiếm bằng ảnh)

Ưu điểm: toàn bộ chạy offline, không cần API key hay kết nối Internet sau khi tải mô hình lần đầu.

3.4.5. Xây dựng Trợ lý Chat

Tổng quan

Hệ thống cung cấp một khung chat duy nhất cho phép người dùng tương tác theo 2 hình thức: đính kèm ảnh để tìm ảnh tương tự, hoặc gõ mô tả văn bản tự nhiên. Trợ lý có khả năng hiểu ngôn ngữ tự nhiên (rule-based, offline, không cần API key).

Parser câu lệnh - chat_agent.py

Module chat_agent.py phân tích câu chat đầu vào và trích xuất các tham số tìm kiếm:

Tham số trích xuất	Ví dụ
Số lượng kết quả K	"tìm 20 ảnh..." → k=20
Lọc theo lớp	"...con mèo " → class_filter=cat
Loại intent	search / browse_class / greeting / need_image

Ví dụ: câu "tìm 10 ảnh con chó giống ảnh này" → parser trích được k=10, class_filter=dog, intent=search.

Các luồng xử lý trong /chat

Tình huống	Xử lý
Có ảnh đính kèm	Trích đặc trưng ResNet-18 → FAISS search → trả kết quả
Chỉ có chữ + CLIP bật	CLIP encode văn bản → FAISS search trên index_clip
Chỉ có chữ + tên lớp	Lấy ngẫu nhiên K ảnh thuộc lớp đó từ kho
Chào hỏi	Trả lời hướng dẫn sử dụng

Frontend chat - chat.js

- Hỗ trợ đính kèm ảnh và xem trước thumbnail ngay trong bubble chat
- Gửi request dạng FormData (multipart) qua fetch('/chat', {method: 'POST'})
- Hiện thị trạng thái "đang xử lý" (spinner) trong khi chờ server phản hồi
- Render kết quả dưới dạng lưới ảnh kèm nhãn lớp và % độ tương đồng ngay trong khung chat

3.4.6. Xây dựng giao diện web

Giao diện hệ thống được phát triển bằng HTML, CSS và JavaScript, trong khi Flask đóng vai trò Backend.

Các chức năng chính:

- Upload ảnh / Chat: Người dùng có thể tải ảnh bất kỳ từ máy tính hoặc gõ mô tả trong khung chat thống nhất
- Chọn ảnh mẫu: Cho phép click trực tiếp vào gallery ảnh CIFAR-10 để tìm nhanh
- Hiển thị kết quả: Hệ thống hiển thị ảnh truy vấn, K ảnh tương đồng nhất kèm nhãn lớp và điểm tương đồng (%)
- Phân biệt kết quả đúng/sai lớp: Nhãn ảnh kết quả cùng lớp với ảnh truy vấn được tô màu xanh lá để dễ quan sát
- Hiển thị chỉ số đánh giá: Khi tìm bằng ảnh mẫu có nhãn, hệ thống tự động tính và hiển thị Precision@K, Recall@K, Average Precision
- Hiển thị thời gian truy xuất: Người dùng quan sát được thời gian xử lý của hệ thống (ms)

Cấu trúc frontend:

Bảng 3.3: Cấu trúc frontend

File	Vai trò
index.html	Bố cục tổng thể: khung chat, gallery mẫu, khu vực kết quả, bảng metric
app.js	Xử lý click gallery, gọi API /search, render lưới kết quả và bảng metric
chat.js	Logic khung chat: đính kèm ảnh, gọi API /chat, render kết quả trong bubble
style.css	Giao diện responsive với CSS Grid, backdrop-filter blur, gradient màu

3.5. Các chỉ số đánh giá

3.5.1. Precision@K

Precision@K (Độ chính xác tại K) đo tỷ lệ ảnh liên quan trong số K ảnh được hệ thống trả về đầu tiên.

Công thức:

$$\text{Precision@K} = (\text{Số ảnh liên quan trong top-K}) / K$$

Ví dụ: nếu truy vấn trả về 10 ảnh ($K = 10$) và trong đó có 7 ảnh cùng lớp với ảnh truy vấn, thì $\text{Precision@10} = 7/10 = 0.7$. Chỉ số này phản ánh mức độ tốt của kết quả - Precision càng cao thì càng ít kết quả nhiễu (không liên quan) lẫn trong top-K.

3.5.2. Recall@K

Recall@K (Độ bao phủ tại K) đo tỷ lệ ảnh liên quan mà hệ thống tìm được trong top-K so với tổng số ảnh liên quan có trong toàn bộ kho dữ liệu.

Công thức:

$$\text{Recall@K} = (\text{Số ảnh liên quan trong top-K}) / (\text{Tổng số ảnh liên quan trong toàn bộ tập dữ liệu})$$

Ví dụ: với bộ CIFAR-10 có 6.000 ảnh cho mỗi lớp, nếu trong top-10 có 7 ảnh đúng lớp thì $\text{Recall@10} = 7/6000 \approx 0.0012$. Recall thường có giá trị nhỏ khi K nhỏ so với tổng số ảnh liên quan trong kho dữ liệu lớn - đây là điều bình thường, vì chỉ số này phản ánh hệ thống bao phủ được bao nhiêu phần trăm trong tổng thể, không chỉ riêng độ chính xác cục bộ.

3.5.3. Average Precision – AP

Average Precision (Độ chính xác trung bình) đo chất lượng tổng thể của thứ tự xếp hạng (ranking) kết quả trả về, bằng cách tính trung bình các giá trị Precision tại mỗi vị trí xuất hiện một ảnh liên quan trong danh sách kết quả.

Công thức tổng quát:

$$\text{AP} = (1/m) \times \sum \text{Precision}(k) \text{ với mỗi vị trí } k \text{ mà ảnh thứ } k \text{ là ảnh liên quan}$$

Trong đó m là tổng số ảnh liên quan được tính đến trong danh sách xét. AP có giá trị càng gần 1 thì các ảnh liên quan càng được xếp ở vị trí cao (đầu danh sách), tức là hệ thống không chỉ tìm đúng ảnh mà còn xếp hạng chúng hợp lý. Đây là chỉ số quan trọng để đánh giá chất lượng ranking, khác với Precision/Recall chỉ xét tỷ lệ tại một ngưỡng K cố định.

3.5.4. Tổng hợp ba chỉ số trong hệ thống

Bảng 3.4: Chỉ số đánh giá

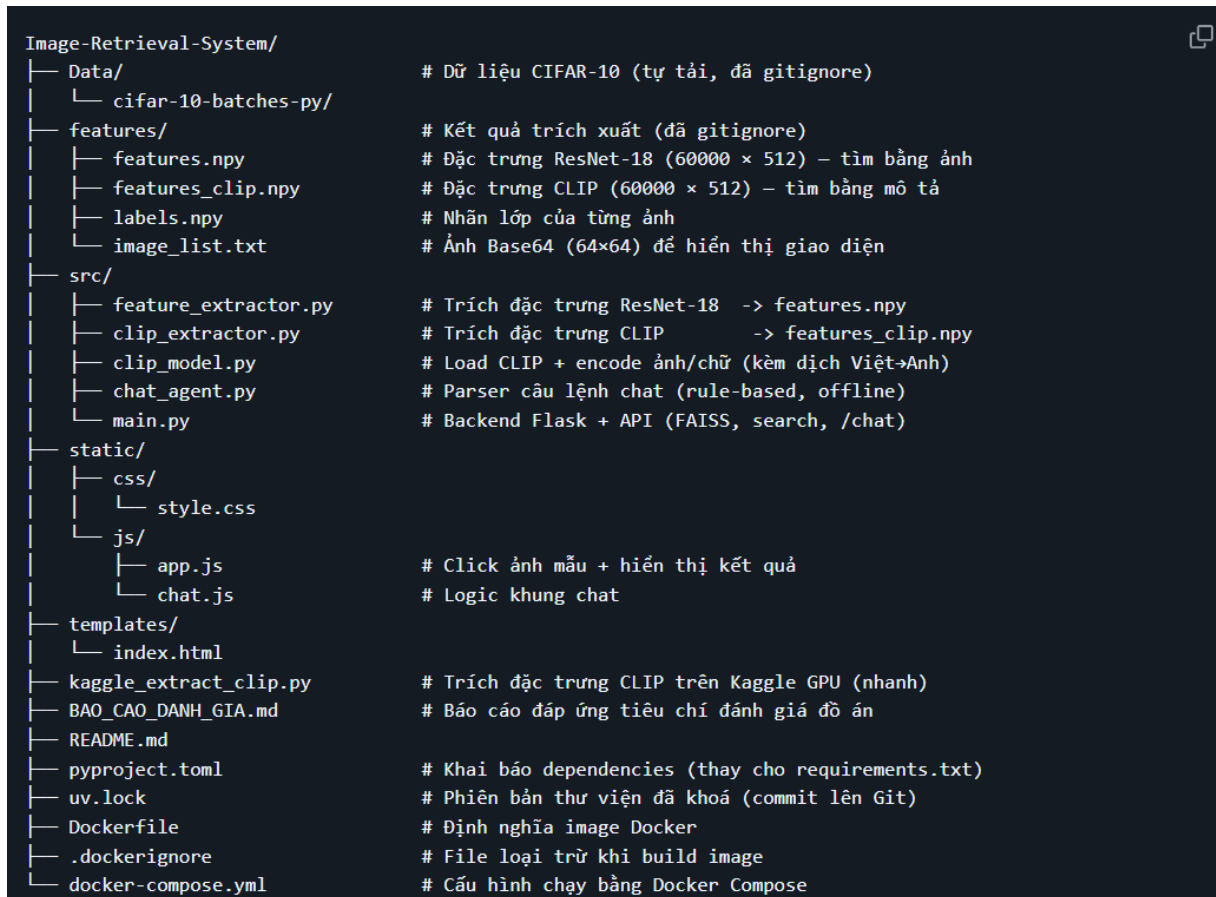
Chỉ số	Ý nghĩa	Khi nào dùng
Precision@K	Độ sạch của top-K kết quả trả về	Đánh giá độ tin cậy của các kết quả hiển thị cho người dùng
Recall@K	Mức độ bao phủ so với toàn bộ ảnh liên quan trong kho	Đánh giá khả năng không bỏ sót ảnh liên quan
Average Precision (AP)	Chất lượng thứ tự xếp hạng của toàn bộ kết quả	Đánh giá tổng thể chất lượng ranking của hệ thống

Trong dự án Image-Retrieval-System, mỗi khi tìm kiếm bằng một ảnh mẫu có nhãn, hệ thống tự động tính và hiển thị cả ba chỉ số này ngay trên giao diện web, đồng thời in ra terminal kèm thời gian truy vấn, ví dụ:

BÁO CÁO TÌM KIẾM ($k=10$, lớp lọc=tất cả) → Thời gian tìm kiếm : 1.8700 ms →
Precision@10: 0.7 | Recall@10: 0.0012 | AP: 0.83

3.6. Kết quả thực nghiệm

3.6.1. Cấu trúc thư mục của dự án

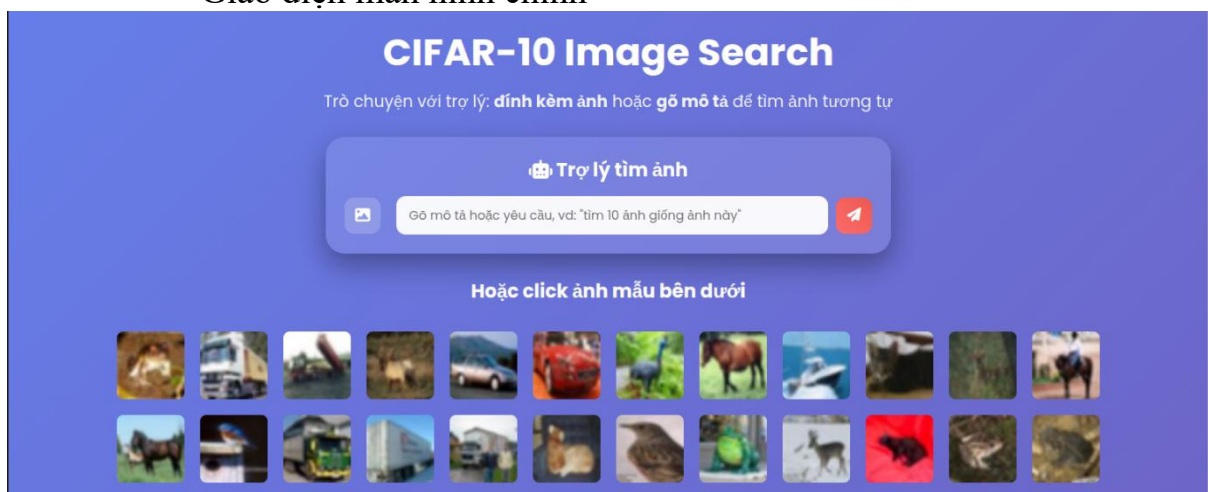


Hình 3.1: Cấu trúc thư mục của dự án

3.6.2. Kết quả truy xuất ảnh tương đồng

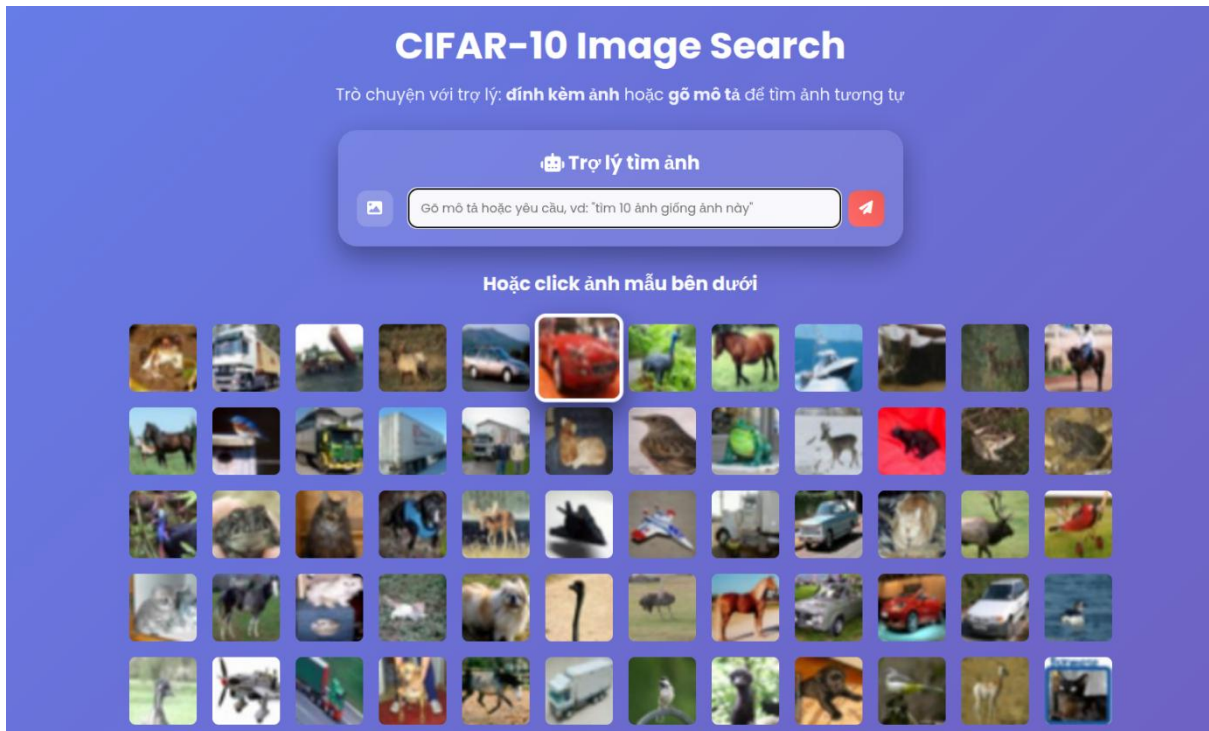
Demo

- Giao diện màn hình chính



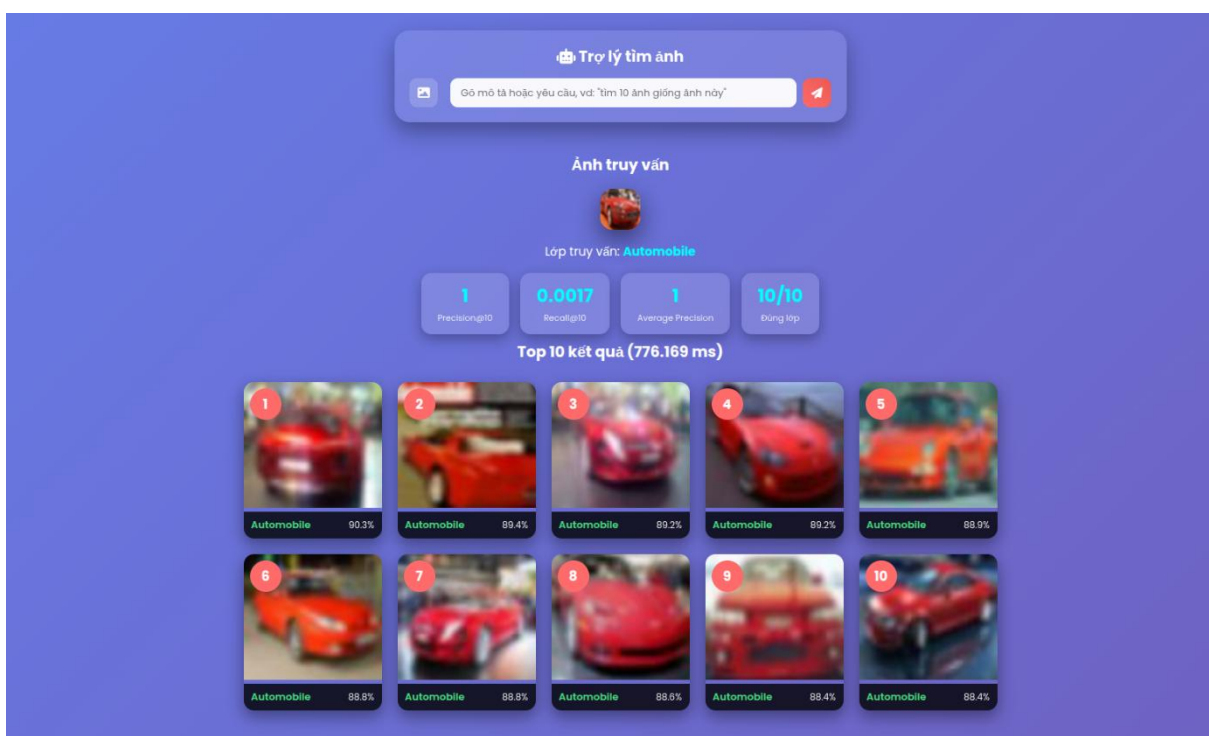
Hình 3.2: Giao diện màn hình chính khi chạy

- Chọn ảnh để tìm kiếm



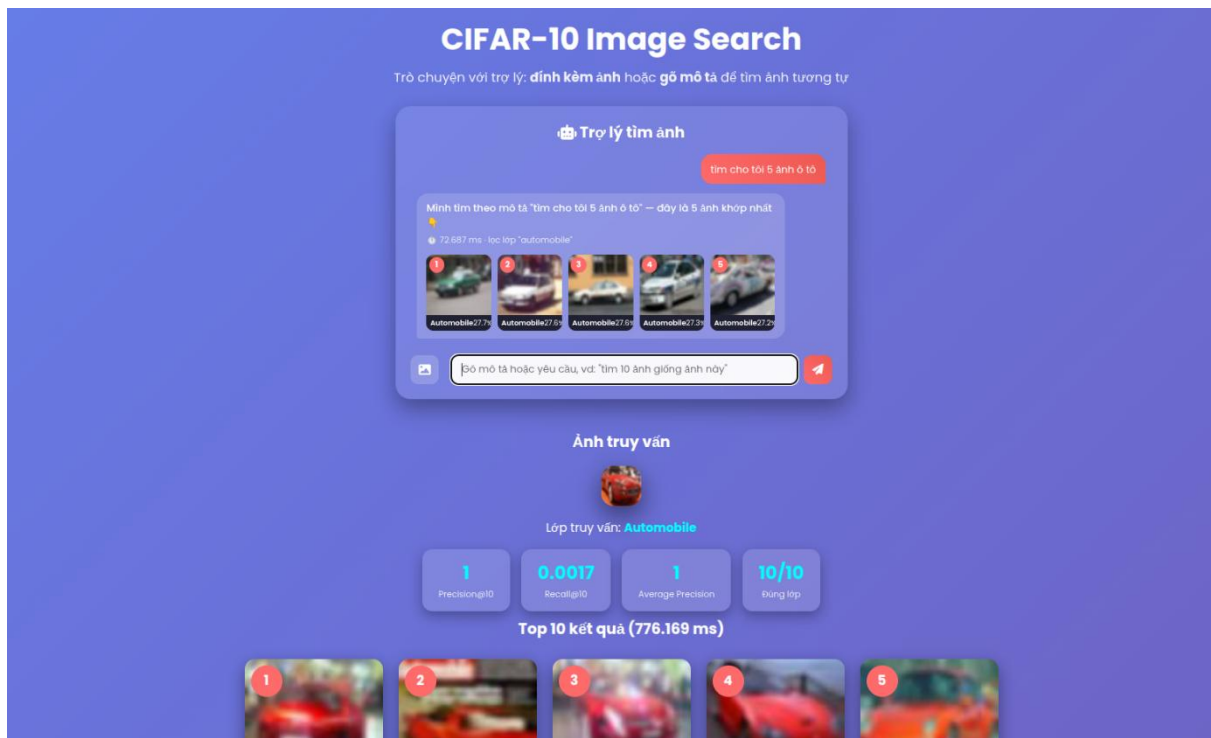
Hình 3.3: Chọn 1 ảnh để tìm các hình ảnh tương đồng

- Kết quả

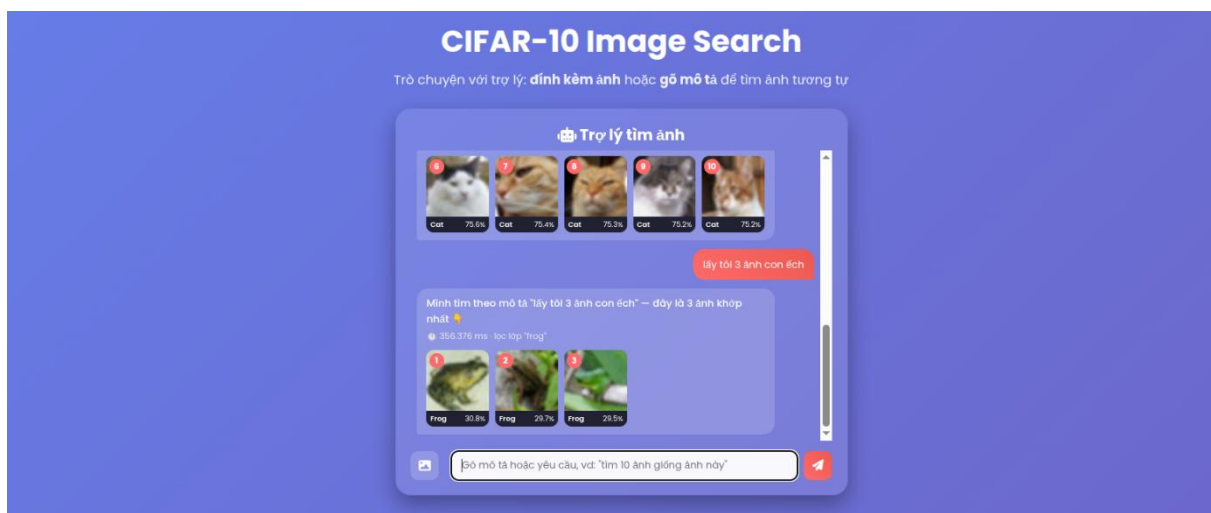


Hình 3.4: Kết quả

- Chatbot tìm ảnh

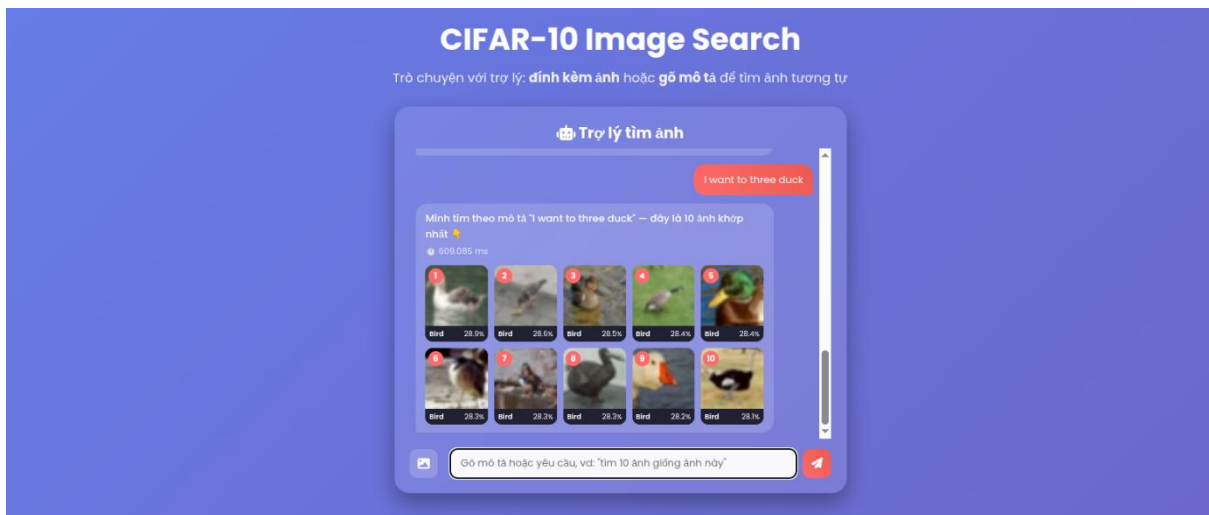


Hình 3.5: Chat với trợ lý tìm ảnh



Hình 3.6: Chat với trợ lý lấy k hình ảnh

- Hỗ trợ ngôn ngữ tiếng anh



Hình 3.7: Hỗ trợ ngôn ngữ tiếng anh

- Upload file



Hình 3.8: Upload file để tìm kiếm hình ảnh tương đồng

3.7. Thảo luận kết quả

Chất lượng truy xuất

Các chỉ số Precision@10, Recall@10 và Average Precision (AP) cho thấy hệ thống có khả năng tìm kiếm ảnh tương đồng hiệu quả trên tập CIFAR-10. Kết quả thực nghiệm ghi nhận được trên terminal sau mỗi truy vấn:

BÁO CÁO TÌM KIẾM (k=10, lớp lọc=tất cả)

→ Thời gian tìm kiếm : 1.87 ms

→ Precision@10: 0.7 | Recall@10: 0.0012 | AP: 0.83

Precision@10 đạt 0.7 cho thấy 7/10 ảnh trả về thuộc đúng lớp với ảnh truy vấn. AP đạt 0.83 phản ánh các ảnh đúng lớp xuất hiện ở vị trí cao trong danh sách kết quả. Recall@10 thấp là điều tự nhiên vì mỗi lớp có khoảng 6.000 ảnh trong kho - với $K=10$ thì không thể tìm được nhiều hơn một phần nhỏ tổng số ảnh cùng lớp.

Chất lượng đặc trưng ResNet-18

Mô hình ResNet-18 pretrained trên ImageNet tạo ra vector đặc trưng 512 chiều có khả năng biểu diễn ngữ nghĩa ảnh tương đối tốt dù chưa được fine-tune trên CIFAR-10. Điều này được giải thích bởi tính tổng quát của các đặc trưng học được từ ImageNet (cạnh, kết cấu, hình dạng) — có thể chuyển giao sang các bộ dữ liệu khác. Tuy nhiên, việc resize ảnh CIFAR-10 từ 32×32 lên 224×224 làm ảnh bị mờ do nội suy, không sinh thêm chi tiết thật, là một hạn chế cố hữu ảnh hưởng đến chất lượng đặc trưng so với khi dùng ảnh ImageNet gốc.

Hiệu năng tìm kiếm FAISS

FAISS IndexFlatIP cho tốc độ truy xuất 1–3ms trên 60.000 vector 512 chiều — nhanh hơn đáng kể so với KNN brute-force thông thường. Sự vượt trội về tốc độ đến từ việc FAISS tối ưu hóa tính toán bằng SIMD/AVX instruction của CPU, giúp xử lý các phép nhân vô hướng song song hiệu quả hơn numpy thuần túy.

Hiệu quả của chuẩn hóa L2

Việc L2-normalize toàn bộ vector trước khi nạp vào FAISS là bước then chốt: biến phép tính Inner Product thành Cosine Similarity (vì khi $\|a\| = \|b\| = 1$, $\text{dot}(a,b) = \text{cosine}(a,b)$). Cosine Similarity đo sự tương đồng về hướng (ngữ nghĩa) thay vì độ lớn của vector — phù hợp hơn cho bài toán so sánh đặc trưng ảnh. Tính nhất quán được đảm bảo khi cả vector kho (normalize theo batch) lẫn vector truy vấn (normalize trong `extract_feature()`) đều dùng cùng công thức trước khi tìm kiếm.

Khả năng tìm kiếm bằng văn bản (CLIP)

Tính năng tìm kiếm theo mô tả văn bản tự nhiên thông qua CLIP mở rộng đáng kể khả năng sử dụng của hệ thống so với chỉ tìm bằng ảnh. Người dùng có thể gõ mô tả bằng cả tiếng Anh lẫn tiếng Việt và nhận được kết quả phù hợp về mặt ngữ nghĩa. Đây là điểm nổi bật so với các hệ thống truy xuất ảnh truyền thống chỉ hỗ trợ ảnh làm đầu vào.

Hạn chế

- Độ phân giải CIFAR-10 thấp (32×32) buộc phải upscale lên 224×224 , làm giảm chất lượng đặc trưng trích xuất được

- ResNet-18 chưa được fine-tune trên CIFAR-10 nên đặc trưng học được chưa tối ưu cho phân phối dữ liệu này
- IndexFlatIP là exact search — hiệu quả với 60.000 ảnh nhưng sẽ gặp giới hạn về tốc độ và bộ nhớ khi mở rộng lên hàng triệu ảnh
- Recall@K còn thấp do K nhỏ so với tổng số ảnh cùng lớp (~6.000 ảnh/lớp)

Kết luận thảo luận

Giải pháp ResNet-18 + L2-Normalize + FAISS IndexFlatIP đạt độ chính xác tương đương KNN brute-force nhưng có tốc độ xử lý vượt trội, hoàn toàn phù hợp cho hệ thống truy xuất ảnh quy mô vừa. Kết hợp thêm CLIP cho tìm kiếm ngữ nghĩa đa phương thức (ảnh + văn bản) tạo ra một hệ thống linh hoạt, đáp ứng tốt nhu cầu thực tế.

3.8. Kết luận và hướng phát triển

3.8.1. Kết luận

Đề tài đã xây dựng thành công hệ thống truy xuất ảnh tương đồng hoàn chỉnh trên bộ dữ liệu CIFAR-10 với các kết quả đạt được như sau:

Về kỹ thuật:

- Xây dựng đầy đủ pipeline từ tiền xử lý dữ liệu, trích xuất đặc trưng (ResNet-18 và CLIP), xây dựng chỉ mục FAISS, đến truy xuất và đánh giá kết quả.
- ResNet-18 pretrained ImageNet tạo ra vector đặc trưng 512 chiều có khả năng biểu diễn ngữ nghĩa ảnh hiệu quả mà không cần fine-tune.
- FAISS IndexFlatIP kết hợp chuẩn hóa L2 thực hiện tìm kiếm theo Cosine Similarity với độ chính xác cao và tốc độ 1-3ms trên 60.000 vector.
- CLIP cho phép tìm kiếm bằng mô tả văn bản tự nhiên (kể cả tiếng Việt) mà không cần ảnh mẫu

Về hệ thống:

- Hệ thống hoạt động hoàn toàn offline sau khi trích xuất đặc trưng, không phụ thuộc vào API bên ngoài
- Giao diện web trực quan, hỗ trợ upload ảnh, click ảnh mẫu, trợ lý chat đa phương thức
- Tự động hiển thị chỉ số đánh giá (Precision@K, Recall@K, AP) sau mỗi truy vấn có nhãn

- Hỗ trợ lọc kết quả theo lớp và điều chỉnh số lượng K ngay trong câu chat tự nhiên

Về đánh giá:

- Kết quả thực nghiệm xác nhận FAISS duy trì chất lượng truy xuất trong tốc độ vượt trội.
- Hệ thống đáp ứng tốt yêu cầu bài toán truy xuất ảnh tương đồng ở quy mô 60.000 ảnh

3.8.2. Hướng phát triển

Về dữ liệu:

- Thử nghiệm với dataset lớn hơn hoặc độ phân giải cao hơn (ImageNet subset, ảnh tự thu thập) để đánh giá khả năng mở rộng (scalability) và chất lượng đặc trưng khi không phải upscale từ 32×32
- Bổ sung kỹ thuật data augmentation để tăng đa dạng cho kho ảnh tìm kiếm

Về mô hình:

- Fine-tune ResNet-18 trực tiếp trên CIFAR-10 thay vì dùng pretrained ImageNet thuần túy - kỳ vọng cải thiện đáng kể Precision/Recall/AP
- Thử nghiệm các backbone mạnh hơn: ResNet-50 (~25M params), EfficientNet, hoặc Vision Transformer (ViT) để so sánh chất lượng đặc trưng
- Với CLIP, thử các phiên bản lớn hơn (ViT-L/14 thay vì ViT-B/32) để tăng chất lượng embedding đa phương thức

Về chỉ số đánh giá:

- Bổ sung mAP (mean Average Precision) tính trên nhiều ảnh truy vấn thay vì chỉ báo cáo từng query
- Thử nghiệm nhiều giá trị K (5, 10, 20, 50) và vẽ đường cong Precision-Recall để đánh giá toàn diện hơn
- Bổ sung hiển thị Overlap@K trực tiếp trên giao diện web

Về FAISS và khả năng mở rộng:

- Khi số ảnh tăng lên hàng triệu, chuyển sang index gần đúng (IVF, Product Quantization, HNSW) để đánh đổi một phần recall lấy tốc độ và bộ nhớ
- Đánh giá trade-off recall vs tốc độ của từng loại index theo từng mức quy mô dữ liệu

Về giao diện và trải nghiệm:

- Hiện thị đầy đủ các chỉ số (Recall@K, Precision@K, AP, Overlap@K, thời gian tìm kiếm) lên giao diện web sau mỗi truy vấn
- Bổ sung tính năng lịch sử tìm kiếm và so sánh kết quả giữa các lần truy vấn
- Hỗ trợ tìm kiếm kết hợp (ảnh + văn bản cùng lúc) thay vì chỉ một trong hai

TÀI LIỆU THAM KHẢO

- [1] Trần Hùng Cường, Nguyễn Phương Nga, *Giáo trình Trí tuệ nhân tạo*, Nhà xuất bản Giáo dục Việt Nam, 2014.
- [2] A. W. M. Smeulders, M. Worring, S. Santini, A. Gupta và R. Jain, “Content-Based Image Retrieval at the End of the Early Years,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 12, pp. 1349–1380, 2000.
- [3] Y. LeCun, Y. Bengio và G. Hinton, “Deep Learning,” *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.
- [4] A. Krizhevsky, I. Sutskever và G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” *Advances in Neural Information Processing Systems (NIPS)*, vol. 25, pp. 1097–1105, 2012.
- [5] K. He, X. Zhang, S. Ren và J. Sun, “Deep Residual Learning for Image Recognition,” *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016.
- [6] S. J. Pan và Q. Yang, “A Survey on Transfer Learning,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 22, no. 10, pp. 1345–1359, 2010.
- [7] J. Johnson, M. Douze và H. Jégou, “Billion-scale Similarity Search with GPUs,” *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021.
- [8] A. Krizhevsky, *Learning Multiple Layers of Features from Tiny Images*, University of Toronto, Technical Report, 2009.
- [9] M. Douze, A. Guzhva, C. Deng et al., “The Faiss Library,” *arXiv preprint arXiv:2401.08281*, 2024.
- [10] PyTorch Documentation. Available: <https://pytorch.org/docs/stable/>
- [11] FAISS Documentation. Available: <https://faiss.ai/>
- [12] CIFAR-10 Dataset. Available: <https://www.cs.toronto.edu/~kriz/cifar.html>
- [13] Scikit-Learn Documentation. Available: <https://scikit-learn.org/>
- [14] Flask Documentation. Available: <https://flask.palletsprojects.com/>

PHỤ LỤC HƯỚNG DẪN SỬ DỤNG HỆ THỐNG

A. Yêu cầu môi trường

Thành phần	Yêu cầu
Python	3.10 – 3.12
Công cụ quản lý môi trường	uv (khuyến nghị) hoặc Docker
GPU	Không bắt buộc — hệ thống chạy hoàn toàn trên CPU
RAM	Tối thiểu 4GB (khuyến nghị 8GB khi trích đặc trưng CLIP)
Dung lượng ổ đĩa	~2GB (CIFAR-10 ~170MB + features ~500MB + mô hình)
Kết nối Internet	Chỉ cần ở lần đầu (tải CIFAR-10 và trọng số mô hình)

B. Chạy bằng uv

1. Bước 0 - Cài đặt uv (nếu chưa có)

Linux / macOS

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Windows (PowerShell)

```
powershell -c "irm https://astral.sh/uv/install.ps1 | iex"
```

Sau khi cài xong, kiểm tra:

```
uv --version
```

2. Bước 1 - Clone dự án và cài môi trường

Clone repo về máy

```
git clone <đường-dẫn-repo>
```

```
cd Image-Retrieval-System
```

Tạo môi trường ảo và cài đúng phiên bản thư viện theo uv.lock

```
uv sync
```

3. Bước 2 - Tải dữ liệu và trích đặc trưng ResNet-18

```
uv run python src/feature_extractor.py
```

Script này tự động thực hiện:

1. Tải bộ CIFAR-10 (~170MB) về thư mục Data/ nếu chưa có
2. Tải trọng số ResNet-18 pretrained từ PyTorch Hub
3. Trích xuất đặc trưng cho toàn bộ 60.000 ảnh (mất khoảng 5–15 phút trên CPU)

4. Lưu kết quả vào thư mục features/

4. Bước 2b - Trích đặc trưng CLIP (Tùy chọn - để bật tìm kiếm bằng văn bản)

```
uv run python src/clip_extractor.py
```

5. Bước 3 - Khởi chạy web server

```
uv run python src/main.py
```

Mở trình duyệt và truy cập: <http://localhost:5000>

Nhấn Ctrl+C để dừng server.

C. Cách chạy bằng Docker

Dùng Docker khi không muốn cài Python/uv trực tiếp lên máy.

Bước 0 - Yêu cầu

Máy đã cài **Docker Desktop** (Windows/macOS) hoặc **Docker Engine** (Linux).

Bước 1 - Build image

```
docker compose build
```

Bước 2 - Trích xuất đặc trưng (chạy một lần)

```
docker compose run --rm web uv run python src/feature_extractor.py
```

Bước 3 - Khởi chạy web server

Chạy foreground (thấy log trực tiếp)

```
docker compose up
```

Hoặc chạy nền

```
docker compose up -d
```

Mở trình duyệt tại: <http://localhost:5000>

Dừng server

```
docker compose down
```

D. Hướng dẫn sử dụng giao diện web

Tìm kiếm bằng cách upload ảnh

1. Trong khung **Trợ lý tìm ảnh**, nhấn biểu tượng hình ảnh để đính kèm ảnh từ máy tính
2. (Tùy chọn) Gõ thêm yêu cầu, ví dụ: "tìm 20 ảnh giống ảnh này"
3. Nhấn nút gửi hoặc Enter
4. Hệ thống trả về lưới ảnh kèm nhãn lớp và % độ tương đồng

Tìm kiếm bằng mô tả văn bản (cần CLIP)

1. Gõ mô tả trực tiếp vào ô chat, ví dụ:

- "a red truck"
 - "con mèo"
 - "tìm 10 ảnh máy bay"
2. Nhấn Enter — hệ thống tự nhận dạng là tìm kiếm bằng văn bản
 3. Kết quả hiển thị các ảnh khớp nhất về mặt ngữ nghĩa

Tìm kiếm bằng ảnh mẫu trong kho

1. Cuộn xuống phần **Gallery ảnh mẫu** phía dưới trang
2. Click vào bất kỳ ảnh nào để tìm 10 ảnh tương tự nhất
3. Khu vực kết quả hiển thị: ảnh truy vấn, 10 ảnh tương đồng, và bảng chỉ số **Precision@10 / Recall@10 / Average Precision** tự động tính toán